ICPC Team Reference Document

Jagiellonian University, Kraków — team UZI

1 Contest **1**
- 1.1 Template 1
- 1.2 Compile & stress test 1
- 1.3 Fast input 1
- 1.4 Limits ($\approx 10^8$ simple operations/s) 1
- 1.5 Before submitting / after WA 1
- 1.6 When stuck: ideas to try 1

2 Mathematics **2**
- 2.1 Modular integer (998244353) 2
- 2.2 Binomial coefficients 2
- 2.3 Power, extended gcd, inverse, CRT 2
- 2.4 Sieve, factorization, divisors, phi 2
- 2.5 Exact integer root and log 3
- 2.6 Miller–Rabin (64-bit) 3
- 2.7 Pollard rho 3
- 2.8 Matrix power 3
- 2.9 Gaussian elimination mod p 3
- 2.10 Determinant mod p 3
- 2.11 Quadratic extension field (with $\sqrt{5}$) 3
- 2.12 XOR basis 4
- 2.13 NTT (convolution mod 998244353) 4
- 2.14 FFT (exact integer convolution) 4
- 2.15 Berlekamp–Massey, k-th term 4
- 2.16 Lagrange interpolation 4
- 2.17 Discrete log, floor sum 5
- 2.18 Number theory facts 5
- 2.19 Combinatorics 5
- 2.20 Sums 5
- 2.21 Games 5

3 Data structures **5**
- 3.1 Disjoint set union 5
- 3.2 Fenwick tree 5
- 3.3 Segment tree (point update) 6
- 3.4 Lazy segment tree (generic) 6
- 3.5 Sparse table 6
- 3.6 Monotonic stack, sliding window 6
- 3.7 Ordered set, hash map 6
- 3.8 Li Chao tree (lines, min) 7
- 3.9 Implicit treap 7
- 3.10 DSU with rollback 7
- 3.11 Mo's algorithm 7
- 3.12 Mo's algorithm with rollback 7
- 3.13 Persistent segment tree 8
- 3.14 Segment tree beats (chmin, sum) 8
- 3.15 Records segment tree 8
- 3.16 Dynamic segment tree 8
- 3.17 2D Fenwick (generic prefix) 8
- 3.18 CDQ divide and conquer 8
- 3.19 Ideas 9

4 Graphs **9**
- 4.1 Dijkstra, 0-1 BFS, Bellman–Ford 9
- 4.2 Topological sort 9
- 4.3 SCC (Tarjan) and 2-SAT 9
- 4.4 Bridges and articulation points 10
- 4.5 LCA (binary lifting) 10
- 4.6 Heavy-light decomposition 10
- 4.7 Centroid decomposition 10
- 4.8 Virtual tree 10
- 4.9 Small-to-large (DSU on tree) 11
- 4.10 Block-cut tree 11
- 4.11 Functional graph 11
- 4.12 Binary lifting with costs 11
- 4.13 Link-cut tree 11
- 4.14 Euler path 12
- 4.15 Bipartite matching (Kuhn) 12
- 4.16 Max flow (Dinic) 12
- 4.17 Min-cost flow 12
- 4.18 Hungarian algorithm 12
- 4.19 Facts 13

5 Strings **13**
- 5.1 Prefix function, Z-function 13
- 5.2 Polynomial hashing 13
- 5.3 Manacher 13
- 5.4 Suffix array + LCP 13
- 5.5 Aho–Corasick 13
- 5.6 Binary trie (max XOR) 14
- 5.7 XOR trie with a bound 14
- 5.8 Counting strings by prefix function 14
- 5.9 Suffix automaton 14
- 5.10 Ideas 14

6 Geometry **14**
- 6.1 Point 14
- 6.2 Segments and lines 15
- 6.3 Polygons, convex hull 15
- 6.4 Polar angle sort 15
- 6.5 Circles 15
- 6.6 Minimum enclosing circle 15
- 6.7 Point in convex polygon, closest pair 15
- 6.8 Polygon inside polygon 16
- 6.9 Angular sweep counting 16
- 6.10 Formulas 16

7 DP and techniques **16**
- 7.1 DP patterns 16
- 7.2 Knuth optimisation 16
- 7.3 Aliens trick (exactly K items) 17
- 7.4 Re-rooting 17
- 7.5 Binary / ternary search 17
- 7.6 Bits, random, misc 17
- 7.7 Numerical integration 17
- 7.8 DP optimisations 17
- 7.9 STL reminders and pitfalls 17

1 Contest

Template

0-contest/template.cpp

```cpp
// #pragma GCC optimize("O3,unroll-loops")  // heavy loops
// #pragma GCC target("avx2")  // bitsets/SIMD; judge CPU!
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#ifdef LOCAL
#define dbg(...) \
  cerr << "[" #__VA_ARGS__ "] ", dbg_(__VA_ARGS__)
template <class... T> void dbg_(T... a) {
  ((cerr << a << ' '), ...); cerr << endl; }
#else
#define dbg(...)
#endif

int main() {
  cin.tie(0)->sync_with_stdio(0);
  int t = 1; // cin >> t;   (multitest: RESET globals!)
  while (t--) {
  }
}
```

Compile & stress test

```
# compile (debug):  sanitizers find out-of-bounds / overflow
g++ -std=gnu++20 -O2 -Wall -Wextra -Wshadow -DLOCAL \
  -fsanitize=address,undefined -D_GLIBCXX_DEBUG a.cpp -o a
# compile (fast):
g++ -std=gnu++20 -O2 a.cpp -o a && ./a < in
# stress test: gen.cpp reads seed from argv[1]
for i in $(seq 1 1000); do
  ./gen $i > in; ./a < in > o1; ./brute < in > o2
  cmp -s o1 o2 || { echo "DIFF seed $i"; cat in; break; }
done
```

Fast input

0-contest/fastio.cpp

```cpp
// Fast input: 2x faster than unsynced cin (2e6 ints: 60 ms
// vs 120 ms). Use when input is > 5e6 numbers.
// Do not mix with cin/scanf; not for interactive problems.
char buf[1 << 16]; int bufLen = 0, bufPos = 0;
int gc() {
  if (bufPos == bufLen) {
    bufLen = fread(buf, 1, sizeof buf, stdin); bufPos = 0;
    if (bufLen <= 0) return -1;
  }
  return buf[bufPos++]; }
ll readInt() {
  int c = gc(); bool neg = false;    // '-' right before digits
  while (c < '0' || c > '9') {
    if (c == -1) return 0;           // end of input
    neg = c == '-'; c = gc(); }
  ll x = 0;
  for (; c >= '0' && c <= '9'; c = gc())
    x = x * 10 + (c - '0');
  return neg ? -x : x; }
string readStr() {                   // next non-space token
  int c = gc();
  while (c != -1 && isspace(c)) c = gc();
  string s;
  for (; c != -1 && !isspace(c); c = gc()) s += char(c);
  return s; }
```

Limits ($\approx 10^8$ simple operations/s)

$n \le$ 10–11	$n!$, $n! \cdot n$	$n \le 10^5$	$n\sqrt{n}$
$n \le 20$	$2^n n$	$n \le 10^6$	$n \log n$
$n \le 500$	n^3	$n \le 10^8$	n (simple)
$n \le 5000$	n^2	$n \le 10^{18}$	$\log n$, $\sqrt[3]{n}$

`int` $< 2.1 \cdot 10^9$, `ll` $< 9.2 \cdot 10^{18}$, `__int128` $< 1.7 \cdot 10^{38}$. `double`: 15 digits; `long double`: 18 digits.

Before submitting / after WA

- Re-read the statement: output format, 0/1-indexing, "modulo", multiple test cases (reset all global state!), limits of *every* input.
- Overflow: `int*int`, sums, `1<<40` (use `1LL<<`), `INF` too small / `INF+INF`.
- Edge cases: $n = 1$, $n = 0$, all equal, already sorted, negative numbers, maximum test (time!), disconnected graph, self-loops, multi-edges.
- Uninitialised variables, `<` vs `<=`, reading the whole input in multitest even when the answer is known early.
- Recursion depth 10^5–10^6 may crash on an 8 MB stack. Locally: `ulimit -s unlimited`. Judges (DOMjudge) usually allow stack = memory limit; if unsure, go iterative / BFS order.
- Precision: compare with `EPS`, print `setprecision(10)`; avoid `sqrt`/division in integer geometry.
- TLE: `endl` in loops (use `'\n'`), `map`→sort+vector, pass vectors by reference, `memset` per test case, `#pragma GCC optimize("O3")`.
- Write a brute force + generator and stress test (above). Small cases also reveal formulas: print answers for $n = 1..10$ and look for a pattern.

When stuck: ideas to try

- Brute force small cases and look at the numbers (pattern, OEIS-like formula, monotonicity ⇒ binary search on the answer).
- Sort + greedy; prove by exchange argument. Process in reverse (deletions → insertions with DSU). Fix one element / the maximum / the answer value and count the rest.
- Contribution technique: $\sum$ over pairs/subarrays = $\sum$ over elements

of (value × number of structures where it matters).
- Complement counting; inclusion–exclusion over "bad" events.
- Invariants and parity (sum mod k, number of inversions, colouring of a grid). Constructive: find a pattern for small n, then induct.
- Model as a graph: states = vertices (BFS/Dijkstra on (position, extra state)); constraints = flow / matching / 2-SAT.
- Small parameter: bitmask over it, $\sqrt{n}$ split (heavy/light), meet in the middle, randomisation (random shuffle, random pivot, hashing, "answer is one of few candidates").
- Sweep line + data structure over events sorted by coordinate.
- Binary search on the answer + greedy/flow check; parallel binary search for many queries. Greedy segmentation with an expensive check: find each segment end by *galloping* (try $1, 2, 4, \ldots$ then binary search), so the total cost stays $O(n \log n) \times$check.
- Objective linear/bilinear/convex in the parameters ⇒ optimum at a corner/vertex: try only the $O(1)$ candidates. Enumerate the $O(1)$ possible "shapes" of an answer and verify each in $O(n)$.
- L1 objectives ($\sum |x_i - x|$, Manhattan) separate per axis; convex in each coordinate ⇒ ternary search / median.
- Sum of piecewise-linear functions at many points: events (slope and intercept deltas at breakpoints), sort, prefix sums, answer by `upper_bound`.
- Memory limit tiny (16–64 MB): no `vector<vector>` for adjacency (use CSR: count degrees, prefix sums, one array), bitsets, `int`/`short` instead of `ll`, process queries offline grouped by vertex, read input in chunks.

2 Mathematics

Modular integer (998244353) 1-math/mint.cpp

When: any answer "modulo p". Change `MOD` for $10^9 + 7$; division needs a prime `MOD`. **Pitfalls:** `mint(x)` reduces any `ll` (negatives OK); compare with `==`, never with `<`.

```
// Modular arithmetic. MOD must be prime for division.
const int MOD = 998244353;
struct mint {
  int v;
  mint(ll x = 0) { v = x % MOD; if (v < 0) v += MOD; }
  mint pow(ll e) const {
    mint r = 1, b = *this;
    for (; e; e >>= 1, b *= b) if (e & 1) r *= b;
    return r; }
  mint inv() const { return pow(MOD - 2); }
  mint& operator+=(mint o) {
    v += o.v; if (v >= MOD) v -= MOD;
    return *this; }
  mint& operator-=(mint o) {
    v -= o.v; if (v < 0) v += MOD;
    return *this; }
  mint& operator*=(mint o) {
    v = (ll)v * o.v % MOD; return *this; }
  mint& operator/=(mint o) { return *this *= o.inv(); }
  friend mint operator+(mint a, mint b) { return a += b; }
  friend mint operator-(mint a, mint b) { return a -= b; }
  friend mint operator*(mint a, mint b) { return a *= b; }
  friend mint operator/(mint a, mint b) { return a /= b; }
  friend bool operator==(mint a, mint b) { return a.v == b.v; }
  friend ostream& operator<<(ostream& o, mint a) {
    return o << a.v; }
};
```

Binomial coefficients 1-math/binom.cpp

How: call `initFact()` once; `N` must exceed the largest n used (e.g. $n + k$ in stars and bars).

```
// Factorials, C(n,k), permutations of a multiset. O(N) init.
const int N = 2e6 + 5;
mint fact[N], ifact[N];
void initFact() {
  fact[0] = 1;
  for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i;
  ifact[N - 1] = fact[N - 1].inv();
  for (int i = N - 1; i > 0; i--) ifact[i - 1] = ifact[i] * i;
}
mint C(int n, int k) {
  if (k < 0 || k > n) return 0;
  return fact[n] * ifact[k] * ifact[n - k]; }
// n! / (c1! c2! ...), n = sum of c
mint multinom(const vi& c) {
  int n = 0; mint r = 1;
  for (int x : c) n += x, r *= ifact[x];
  return r * fact[n]; }
// Lucas (small prime p, ll n): C(n,k) = prod C(n_i, k_i)
// over base-p digits.
```

Power, extended gcd, inverse, CRT 1-math/modpow.cpp

When: modulus not prime, linear congruences. $ax \equiv b \pmod m$: solvable iff $g = \gcd(a, m) \mid b$; then $x = (b/g)\, x_0 \bmod (m/g)$ with x_0 from `egcd(a, m, x0, y0)`. CRT for many congruences: fold pairwise.

```
// Plain ll version (any modulus m <= 3e9; use __int128
// multiplication for m up to 1e18).
ll power(ll b, ll e, ll m) {
  ll r = 1; b %= m; if (b < 0) b += m;
  for (; e; e >>= 1, b = b * b % m) if (e & 1) r = r * b % m;
  return r; }
// a*x + b*y = g = gcd(a, b)
ll egcd(ll a, ll b, ll& x, ll& y) {
  if (!b) { x = 1, y = 0; return a; }
  ll g = egcd(b, a % b, y, x); y -= a / b * x; return g; }
// inverse of a mod m when gcd(a, m) = 1 (m need not be prime)
ll inverse(ll a, ll m) {
  ll x, y; egcd((a % m + m) % m, m, x, y);
  return (x % m + m) % m; }
// 1 + r + ... + r^(k-1) mod m for ANY m (no inverse needed).
ll geo(ll r, ll k, ll m) {
  if (k == 0) return 0;
  if (k % 2) return (1 + r % m * geo(r, k - 1, m)) % m;
  ll h = geo(r, k / 2, m);
  return (h + h * power(r, k / 2, m)) % m; }
// CRT: x = a (mod m), x = b (mod n), 0<=a<m, 0<=b<n.
// Returns {x, lcm(m,n)} or {-1,-1} if no solution.
pair<ll, ll> crt(ll a, ll m, ll b, ll n) {
  ll x, y, g = egcd(m, n, x, y);
  if ((b - a) % g) return {-1, -1};
  ll l = m / g * n;
  ll t = (__int128)(b - a) / g * x % (n / g);
  ll r = (a + (__int128)t * m) % l;
  return {r < 0 ? r + l : r, l}; }
```

Sieve, factorization, divisors, phi 1-math/sieve.cpp

```
// Linear sieve: spf[x] = smallest prime factor. O(n).
// n = 1e7 -> about 0.1 s, 40 MB (vector<int>).
vi spf, primes;
void sieve(int n) {
  spf.assign(n + 1, 0); primes.clear();
  for (int i = 2; i <= n; i++) {
    if (!spf[i]) spf[i] = i, primes.push_back(i);
    for (int p : primes) {
      if (p > spf[i] || (ll)i * p > n) break;
      spf[i * p] = p;
    }
  }
}
// factorize x <= n in O(log x): {prime, exponent}
vector<pii> factor(int x) {
  vector<pii> f;
  while (x > 1) {
    int p = spf[x], e = 0;
    while (x % p == 0) x /= p, e++;
    f.push_back({p, e});
  }
  return f; }
// trial division for x <= 1e12..1e14: O(sqrt x)
vector<pair<ll, int>> factorSlow(ll x) {
  vector<pair<ll, int>> f;
  for (ll p = 2; p * p <= x; p++) if (x % p == 0) {
    int e = 0; while (x % p == 0) x /= p, e++;
    f.push_back({p, e}); }
  if (x > 1) f.push_back({x, 1});
  return f; }
// all divisors from factorization (unsorted)
vector<ll> divisors(const vector<pair<ll, int>>& f) {
  vector<ll> d{1};
  for (auto [p, e] : f) {
    int s = sz(d);
    for (ll i = 0, q = p; i < e; i++, q *= p)
      for (int j = 0; j < s; j++) d.push_back(d[j] * q);
  }
  return d; }
// Euler phi from factorization: n * prod (1 - 1/p)
ll phi(ll n, const vector<pair<ll, int>>& f) {
  for (auto [p, e] : f) n = n / p * (p - 1);
  return n; }
// all divisors d of n with phi(d), from n's factorization.
// gcd sums: sum_j gcd(x, a_j) = sum_{d | x} phi(d) * cnt[d],
// cnt[d] = #{j : d | a_j}  (because n = sum_{d | n} phi(d)).
vector<pair<ll, ll>> divPhi(const vector<pair<ll, int>>& f) {
  vector<pair<ll, ll>> d{{1, 1}};
  for (auto [p, e] : f) {
    int s = sz(d); ll q = 1, ph = 1;
```

```
    for (int i = 1; i <= e; i++) {
      ph = i == 1 ? p - 1 : ph * p; q *= p;
      for (int j = 0; j < s; j++)
        d.push_back({d[j].first * q, d[j].second * ph});
    }
  }
  return d; }
```

Exact integer root and log

1-math/iroot.cpp

When: k-th roots and logs of huge numbers (pow/log in doubles are off by one near perfect powers).

```
// Exact floor(x^(1/k)) for 0 <= x < 2^63, k >= 1 (powl alone
// is off by one near perfect powers). O(k) per check.
ll iroot(ll x, int k) {
  if (k == 1 || x < 2) return x;
  auto ok = [&](ll r) {                  // r^k <= x ?
    __int128 p = 1;
    for (int i = 0; i < k; i++) if ((p *= r) > x) return false;
    return true; };
  ll r = powl(x, 1.0L / k);
  while (r > 0 && !ok(r)) r--;
  while (ok(r + 1)) r++;
  return r; }
// floor(log_b x) for x >= 1, b >= 2, exact
int ilog(ll x, ll b) {
  int k = 0;
  for (__int128 p = b; p <= x; p *= b) k++;
  return k; }
```

Miller–Rabin (64-bit)

1-math/millerrabin.cpp

```
// Deterministic primality test for all n < 2^64.
using ull = unsigned long long;
ull mulmod(ull a, ull b, ull m) {
  return (__uint128_t)a * b % m; }
ull powmod(ull b, ull e, ull m) {
  ull r = 1;
  for (; e; e >>= 1, b = mulmod(b, b, m))
    if (e & 1) r = mulmod(r, b, m);
  return r; }
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull d = n - 1; int s = __builtin_ctzll(d); d >>= s;
  for (ull a : {2, 325, 9375, 28178, 450775, 9780504,
                1795265022}) {
    ull x = powmod(a % n, d, n); int i = s;
    while (x != 1 && x != n - 1 && a % n && i--)
      x = mulmod(x, x, n);
    if (x != n - 1 && i != s) return false;
  }
  return true; }
```

Pollard rho

1-math/pollard.cpp

When: factorise numbers up to 10^{18} (trial division would need 10^9 steps). **How:** rho finds a non-trivial divisor, recurse on both parts. Result unsorted: sort it, then group equal primes.

```
// Pollard rho: prime factors of n < 2^64 (with repetition,
// unsorted), about O(n^(1/4)). Needs millerrabin.cpp.
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull v) { return mulmod(v, v, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = mulmod(prd, max(x, y) - min(x, y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n); }
vector<ull> factorize(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factorize(x), r = factorize(n / x);
  l.insert(l.end(), all(r));
  return l; }
```

Matrix power

1-math/matrix.cpp

When: linear recurrence / DP with fixed transitions and huge n (state size $k \le 100$); walks of length $n = A^n$. **How:** state vector $v_{n+1} = Mv_n$, answer $M^n v_0$. Prefix sums: add an accumulator state.

```
// Matrix power mod MOD: k x k matrix, O(k^3 log e).
// Linear recurrence f(n) = sum c_i f(n-i) -> companion matrix.
using Mat = vector<vector<ll>>;
Mat mul(const Mat& a, const Mat& b) {
  int n = sz(a), m = sz(b[0]), k = sz(b);
  Mat c(n, vector<ll>(m));
  for (int i = 0; i < n; i++)
    for (int t = 0; t < k; t++) if (a[i][t])
      for (int j = 0; j < m; j++)
        c[i][j] = (c[i][j] + a[i][t] * b[t][j]) % MOD;
  return c; }
Mat mpow(Mat a, ll e) {
  Mat r(sz(a), vector<ll>(sz(a)));
  for (int i = 0; i < sz(a); i++) r[i][i] = 1;
  for (; e; e >>= 1, a = mul(a, a)) if (e & 1) r = mul(r, a);
  return r; }
```

Gaussian elimination mod p

1-math/gauss.cpp

When: systems of linear equations, rank, counting solutions ($p^{m-\text{rank}}$). Over GF(2): rows as bitset, subtraction is XOR (same code, 64× faster). Real numbers: pick the row with the largest $|a_{ic}|$ as pivot, compare with EPS.

```
// Solve A x = b mod MOD (prime). a = n rows of m+1 numbers
// (last column = b). Returns -1 if no solution, else rank;
// x = one solution (free variables = 0). O(n m min(n,m)).
// #solutions = MOD^(m - rank). For GF(2) use bitset rows.
int gauss(vector<vector<ll>> a, vector<ll>& x) {
  int n = sz(a), m = sz(a[0]) - 1, r = 0;
  vi where(m, -1);
  for (int c = 0; c < m && r < n; c++) {
    int p = r;
    while (p < n && !a[p][c]) p++;
    if (p == n) continue;
    swap(a[p], a[r]); where[c] = r;
    ll iv = power(a[r][c], MOD - 2, MOD);
    for (ll& v : a[r]) v = v * iv % MOD;
    for (int i = 0; i < n; i++) if (i != r && a[i][c]) {
      ll f = a[i][c];
      for (int j = c; j <= m; j++)
        a[i][j] = ((a[i][j] - f * a[r][j]) % MOD + MOD) % MOD;
    }
    r++;
  }
  for (int i = r; i < n; i++) if (a[i][m]) return -1;
  x.assign(m, 0);
  for (int c = 0; c < m; c++)
    if (where[c] >= 0) x[c] = a[where[c]][m];
  return r; }
```

Determinant mod p

1-math/det.cpp

```
// Determinant mod MOD (prime), entries in [0, MOD). O(n^3)
// #spanning trees (Kirchhoff): det of the Laplacian D - A
// with one row and the same column removed.
ll det(vector<vector<ll>> a) {
  int n = sz(a); ll res = 1;
  for (int c = 0; c < n; c++) {
    int p = c;
    while (p < n && !a[p][c]) p++;
    if (p == n) return 0;
    if (p != c) swap(a[p], a[c]), res = MOD - res;
    res = res * a[c][c] % MOD;
    ll iv = power(a[c][c], MOD - 2, MOD);
    for (int i = c + 1; i < n; i++) if (a[i][c]) {
      ll f = a[i][c] * iv % MOD;
      for (int j = c; j < n; j++)
        a[i][j] = (a[i][j] + (MOD - f) * a[c][j]) % MOD;
    }
  }
  return res; }
```

Quadratic extension field (with $\sqrt{5}$)

1-math/quadfield.cpp

When: Fibonacci/Lucas sums, products or convolutions mod p (Binet needs $\sqrt{5}$, which exists only in this extension when 5 is a non-residue).

```
// F_p[sqrt(W)] for a quadratic NON-residue W (5 is one mod
// 998244353, so sqrt 5 "exists" here). Binet with
// phi = (1 + sqrt5) / 2: F_n = 2 * (phi^n).b. Turns
// Fibonacci sums/convolutions into geometric series.
const mint W = 5;
struct Q {
  mint a, b;                                   // a + b sqrt(W)
  Q operator+(Q o) const { return {a + o.a, b + o.b}; }
  Q operator-(Q o) const { return {a - o.a, b - o.b}; }
  Q operator*(Q o) const {
    return {a * o.a + b * o.b * W, a * o.b + b * o.a}; }
  Q pow(ll e) const {
    Q r{1, 0}, x = *this;
    for (; e; e >>= 1, x = x * x) if (e & 1) r = r * x;
    return r; }
  Q inv() const {
    mint d = (a * a - b * b * W).inv();
    return {a * d, (mint(0) - b) * d}; }
};
```

XOR basis

1-math/xorbasis.cpp

When: max / count / existence of subset XOR. Two bases merge by inserting all vectors of one into the other ($O(60^2)$).

```
// Linear basis over GF(2) for numbers < 2^60.
struct XorBasis {
  ll b[60] = {}; int cnt = 0;
  bool add(ll x) {              // false if x already spanned
    for (int i = 59; i >= 0; i--) if (x >> i & 1) {
      if (!b[i]) { b[i] = x; cnt++; return true; }
      x ^= b[i];
    }
    return false; }
  ll maxXor(ll x = 0) {         // max of x ^ (subset xor)
    for (int i = 59; i >= 0; i--) x = max(x, x ^ b[i]);
    return x; }
  bool has(ll x) {              // x is a subset xor?
    for (int i = 59; i >= 0; i--)
      if (x >> i & 1) { if (!b[i]) return false; x ^= b[i]; }
    return true; }
};  // # distinct subset xors = 2^cnt
```

NTT (convolution mod 998244353)

1-math/ntt.cpp

When: polynomial products, "number of pairs with $a_i+b_j=s$" for all s (convolve count arrays), DP transitions that are convolutions. **Pitfalls:** inputs must be in $[0, \text{MOD})$; other modulus or exact big counts ⇒ FFT below.

```
// Convolution mod 998244353 (= 119*2^23+1, root 3). O(n log n)
// conv of two arrays of length 5e5: about 0.1 s.
void ntt(vi& a, bool invert) {
  int n = sz(a);
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  vi rt(n / 2 + 1);
  for (int len = 1; len < n; len <<= 1) {
    ll w = power(3, (MOD - 1) / (2 * len), MOD);
    if (invert) w = power(w, MOD - 2, MOD);
    rt[0] = 1;
    for (int j = 1; j < len; j++) rt[j] = rt[j - 1] * w % MOD;
    for (int i = 0; i < n; i += 2 * len)
      for (int j = 0; j < len; j++) {
        int u = a[i + j];
        int v = (ll)a[i + j + len] * rt[j] % MOD;
        a[i + j] = u + v < MOD ? u + v : u + v - MOD;
        a[i + j + len] = u - v >= 0 ? u - v : u - v + MOD;
      }
  }
  if (invert) {
    ll ni = power(n, MOD - 2, MOD);
    for (int& x : a) x = x * ni % MOD;
  }
}
vi conv(vi a, vi b) {           // values in [0, MOD)
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, n = 1;
  while (n < s) n <<= 1;
  a.resize(n); b.resize(n);
  ntt(a, false); ntt(b, false);
  for (int i = 0; i < n; i++) a[i] = (ll)a[i] * b[i] % MOD;
  ntt(a, true); a.resize(s);
  return a; }
```

FFT (exact integer convolution)

1-math/fft.cpp

When: exact integer convolution without modulus (results up to 10^{15}), e.g. counting pairs by sum, string matching with wildcards ($\sum(a-b)^2ab$ trick).

```
// Exact convolution of non-negative ints with any modulus
// (here: plain ll result) via complex FFT; safe while every
// result < 1e15 (e.g. n = 2e5, values <= 1e5 is fine).
using cd = complex<double>;
void fft(vector<cd>& a) {
  int n = sz(a), L = __lg(n);
  static vector<complex<long double>> R(2, 1);
  static vector<cd> rt(2, 1);
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acosl(-1) / k);
    for (int i = k; i < 2 * k; i++)
      rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
  }
  vi rev(n);
  for (int i = 0; i < n; i++)
    rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  for (int i = 0; i < n; i++)
    if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for (int j = 0; j < k; j++) {
        cd z = rt[j + k] * a[i + j + k];
        a[i + j + k] = a[i + j] - z; a[i + j] += z;
      }
}
vector<ll> convExact(const vector<ll>& a,
                     const vector<ll>& b) {
  if (a.empty() || b.empty()) return {};
  vector<ll> res(sz(a) + sz(b) - 1);
  int n = 1; while (n < sz(res)) n <<= 1;
  vector<cd> in(n), out(n);
  for (int i = 0; i < sz(a); i++) in[i].real(a[i]);
  for (int i = 0; i < sz(b); i++) in[i].imag(b[i]);
  fft(in);
  for (cd& x : in) x *= x;
  for (int i = 0; i < n; i++)
    out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  for (int i = 0; i < sz(res); i++)
    res[i] = llround(imag(out[i]) / (4 * n));
  return res; }
```

Berlekamp–Massey, k-th term

1-math/bm.cpp

When: the answer for n satisfies an (unknown) linear recurrence: counting with a fixed-width automaton/DP, tilings, $a_n = \sum c_j a_{n-j}$. **How:** brute-force the first $2L+10$ terms (e.g. 500), run `berlekampMassey`, then `linearRec` for n up to 10^{18}. **Pitfalls:** works only modulo a prime; give it enough terms.

```
// Berlekamp-Massey (mod prime MOD): from the first terms s,
// finds the shortest c with s[i] = sum_j c[j] * s[i-1-j].
// Needs 2L terms for a recurrence of order L. O(n^2)
vector<ll> berlekampMassey(const vector<ll>& s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;
  ll b = 1;
  for (int i = 0; i < n; i++) {
    ++m;
    ll d = s[i] % MOD;
    for (int j = 1; j <= L; j++)
      d = (d + C[j] * s[i - j]) % MOD;
    if (!d) continue;
    T = C; ll coef = d * power(b, MOD - 2, MOD) % MOD;
    for (int j = m; j < n; j++)
      C[j] = (C[j] - coef * B[j - m]) % MOD;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (MOD - x) % MOD;
  return C; }
// k-th term (0-indexed) of s[i] = sum_j tr[j] * s[i-1-j]
// given the first sz(tr) terms S. O(L^2 log k)
ll linearRec(const vector<ll>& S, const vector<ll>& tr, ll k) {
  int n = sz(tr);
  auto combine = [&](const vector<ll>& a,
                     const vector<ll>& b) {
    vector<ll> res(2 * n + 1);
    for (int i = 0; i <= n; i++)
      for (int j = 0; j <= n; j++)
        res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
    for (int i = 2 * n; i > n; i--)
      for (int j = 0; j < n; j++)
        res[i - 1 - j] =
            (res[i - 1 - j] + res[i] * tr[j]) % MOD;
    res.resize(n + 1);
    return res; };
  vector<ll> pol(n + 1), e(n + 1);
  pol[0] = e[1] = 1;
  for (k++; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }
  ll res = 0;
  for (int i = 0; i < n; i++)
    res = (res + pol[i + 1] * S[i]) % MOD;
  return res; }
```

Lagrange interpolation

1-math/lagrange.cpp

When: answer is a polynomial in n of known degree d (sums of powers, counting in d nested loops): compute $y_0..y_d$ by brute force, evaluate at n.

```
// P(x) for the polynomial of degree < n with P(i) = y[i],
// i = 0..n-1. O(n). Needs mint + fact/ifact (binom.cpp).
```

```
// Sum 1^k + ... + m^k is a polynomial of degree k+1 in m:
// compute it for m = 0..k+1 and interpolate at m.
mint lagrange(const vector<mint>& y, ll x) {   // x >= 0
  int n = sz(y);
  if (x < n) return y[x];
  vector<mint> pre(n + 1, 1), suf(n + 1, 1);
  for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * (x - i);
  for (int i = n - 1; i >= 0; i--)
    suf[i] = suf[i + 1] * (x - i);
  mint res = 0;
  for (int i = 0; i < n; i++) {
    mint t = y[i] * pre[i] * suf[i + 1] * ifact[i]
             * ifact[n - 1 - i];
    if ((n - 1 - i) & 1) res -= t; else res += t;
  }
  return res; }
```

Discrete log, floor sum

1-math/dlog.cpp

When: $a^x \equiv b$; floor sums appear in counting lattice points under a line and in $\sum \lfloor (ai+b)/m \rfloor$-type formulas.

```
// Discrete log (baby-step giant-step): smallest x >= 0 with
// a^x = b (mod m), gcd(a, m) = 1, m <= 3e9; -1 if none.
// O(sqrt m) time and memory.
ll dlog(ll a, ll b, ll m) {
  a %= m; b %= m;
  ll n = sqrtl(m) + 1;
  unordered_map<ll, ll> big;            // b * a^q -> largest q
  for (ll q = 0, cur = b; q <= n; q++, cur = cur * a % m)
    big[cur] = q;
  ll an = power(a, n, m);
  for (ll p = 1, cur = an; p <= n; p++, cur = cur * an % m)
    if (big.count(cur)) return p * n - big[cur];
  return -1; }
// floor sum: sum_{i=0}^{n-1} floor((a*i + b) / m),
// n, m >= 1, a, b >= 0. O(log). (lattice points under line)
ll floorSum(ll n, ll m, ll a, ll b) {
  ll ans = 0;
  while (true) {
    if (a >= m) ans += n * (n - 1) / 2 * (a / m), a %= m;
    if (b >= m) ans += n * (b / m), b %= m;
    ll yMax = a * n + b;
    if (yMax < m) break;
    n = yMax / m; b = yMax % m; swap(m, a);
  }
  return ans; }
```

Number theory facts

- Primes: $998244353 = 119 \cdot 2^{23} + 1$ (root 3), 10^9+7, 10^9+9, $10^{18}+3$; $2^{61}-1$.
- Fermat: $a^{p-1} \equiv 1 \pmod p$; Euler: $a^{\varphi(m)} \equiv 1$ if $\gcd(a,m) = 1$. For huge exponents: $a^e \equiv a^{e \bmod (p-1)} \pmod p$ ($p \nmid a$).
- Max number of divisors: $\le 10^6$: 240; $\le 10^9$: 1344; $\le 10^{18}$: 103680. $\pi(10^6) = 78498$, $\pi(10^9) \approx 5.08 \cdot 10^7$. Prime gaps < 300 below 10^{18}.
- $\sum_{d|n} \varphi(d) = n$. $\gcd(F_a, F_b) = F_{\gcd(a,b)}$. $\gcd(a,b) = \gcd(a, b-a)$. $\sum_{i \le n} \lfloor n/i \rfloor$ has $O(\sqrt{n})$ distinct values: `for (l=1; l<=n; l=r+1) { r = n/(n/l); ... }`.
- Möbius: $\sum_{d|n} \mu(d) = [n = 1]$; number of coprime pairs $(a,b) \le n$: $\sum_d \mu(d) \lfloor n/d \rfloor^2$.
- Sum of divisors $\sigma(n) = \prod \frac{p^{e+1}-1}{p-1}$; number of divisors $\prod (e+1)$.
- Wilson: $(p-1)! \equiv -1 \pmod p$. Lucas: $\binom{n}{k} \equiv \prod \binom{n_i}{k_i} \pmod p$ (base-p digits). $\binom{n}{k}$ is odd $\iff k \,\&\, n = k$.
- $\sum_j \gcd(x, a_j) = \sum_{d|x} \varphi(d) \cdot \#\{j : d \mid a_j\}$ (`divPhi`). $\sum_{i \le n} \gcd(i,n) = \sum_{d|n} d\,\varphi(n/d)$.
- lcm of an arithmetic progression $a, a+d, \ldots$: divide out $g = \gcd(a,d)$; a prime $q \le n$, $q \nmid d$ divides the terms with index $\equiv -ad^{-1} \pmod q$: take the max exponent there; primes $> n$ divide at most one term (keep the remaining cofactors of all terms).
- Comparing huge numbers/expressions (products of big numbers, polynomial identities): evaluate mod a few random large primes ($\sim 10^{18}$, `mulmod`) or mod $2^{61}-1$; equal everywhere $\Rightarrow$ equal w.h.p.

Combinatorics

- $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $\sum_{i=0}^{n} \binom{i}{k} = \binom{n+1}{k+1}$; $\sum_k \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$ (Vandermonde).
- Stars and bars: $x_1 + \cdots + x_k = n$, $x_i \ge 0$: $\binom{n+k-1}{k-1}$; with $x_i \ge 1$: $\binom{n-1}{k-1}$.
- Catalan $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$: 1, 1, 2, 5, 14, 42, 132, 429, 1430 (bracket sequences, binary trees, triangulations, monotone paths not crossing the diagonal). Paths $(0,0) \to (a,b)$ touching line $y = x+1$: reflect $\Rightarrow \binom{a+b}{a+1}$.
- Derangements $D_n = (n-1)(D_{n-1} + D_{n-2})$, $D_n \approx n!/e$: 1, 0, 1, 2, 9, 44, 265.
- Stirling 2nd kind $S(n,k) = kS(n-1,k) + S(n-1,k-1)$ (partitions into k non-empty sets); surjections $= k!\,S(n,k) = \sum_i (-1)^i \binom{k}{i} (k-i)^n$. Bell: 1, 1, 2, 5, 15, 52, 203, 877.
- Stirling 1st kind (permutations with k cycles): $c(n,k) = (n-1)c(n-1,k) + c(n-1,k-1)$.
- Integer partitions $p(n)$: 1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42; $p(100) \approx 1.9 \cdot 10^8$.
- Inclusion–exclusion: $|\bigcup A_i| = \sum_{\emptyset \ne S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$.
- Burnside: $\#\text{orbits} = \frac{1}{|G|} \sum_g \text{fix}(g)$; necklaces of n beads, k colours (rotations): $\frac{1}{n} \sum_{d|n} \varphi(d) k^{n/d}$.
- Cayley: n^{n-2} labelled trees; forests of k trees rooted at given k vertices: kn^{n-k-1}. Kirchhoff: #spanning trees = any cofactor of $D - A$.
- Expected value is linear: $E[\sum X_i] = \sum E[X_i]$ (indicators!). Geometric: success prob. $p \Rightarrow$ expected $1/p$ tries. Coupon collector: nH_n. $P(A)$ as fraction: $a \cdot b^{-1} \bmod p$.
- Random k-subset of $\{1..n\}$: $E[\max] = \frac{k(n+1)}{k+1}$, $E[\min] = \frac{n+1}{k+1}$. Harmonic numbers mod p: prefix sums of inverses ($inv_i = -(p/i) \cdot inv_{p \bmod i}$).
- Sequences of length m over c values that contain each of k given values: $\sum_i (-1)^i \binom{k}{i} (c-i)^m$.
- Connected labelled graphs: $C_n = 2^{\binom{n}{2}} - \sum_{k=1}^{n-1} \binom{n-1}{k-1} C_k\, 2^{\binom{n-k}{2}}$. For $G(n,p)$ replace $2^{\binom{i}{2}}$ by probabilities: $P_n = 1 - \sum_k \binom{n-1}{k-1} P_k\,(1-p)^{k(n-k)}$; $E[\#\text{components}] = \sum_k \binom{n}{k} P_k (1-p)^{k(n-k)}$.
- Chirp: $ij = \binom{i+j}{2} - \binom{i}{2} - \binom{j}{2}$, so $\sum_i a_i b_j x^{ij}$-type sums become convolutions.

Sums

$\sum i = \frac{n(n+1)}{2}$, $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$, $\sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1}$, $H_n \approx \ln n + 0.5772$, $\sum_{p \le n} 1/p \approx \ln \ln n$.

Games

Sprague–Grundy: $g(v) = \text{mex}\{g(u)\}$ over moves; a sum of games is lost $\iff$ XOR of $g = 0$. Nim: XOR of piles. Staircase Nim: XOR of odd steps. Misère Nim: as normal unless all piles ≤ 1 (then win $\iff$ even number of piles of 1). Typical trick: compute g by brute force for small states, find the pattern.

3 Data structures

Disjoint set union

2-ds/dsu.cpp

```
// Disjoint set union, O(alpha(n)) amortized.
struct DSU {
  vi p;                          // p[root] = -size
  DSU(int n) : p(n, -1) {}
  int find(int x) { return p[x] < 0 ? x : p[x] = find(p[x]); }
  int size(int x) { return -p[find(x)]; }
  bool unite(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (p[a] > p[b]) swap(a, b);
    p[a] += p[b]; p[b] = a;
    return true; }
};
// Kruskal: sort edges by w; if (d.unite(u, v)) cost += w;
```

Fenwick tree

2-ds/fenwick.cpp

```
// Fenwick tree, 0-indexed: point add, prefix sum. O(log n)
// BIT b(n) holds ll; BIT<mint> / BIT<double> for other types.
template <class T = ll> struct BIT {
  int n; vector<T> t;
  BIT(int n) : n(n), t(n + 1) {}
  void add(int i, T v) {
    for (i++; i <= n; i += i & -i) t[i] += v; }
  T sum(int i) {                 // a[0] + ... + a[i-1]
    T s = 0;
    for (; i > 0; i -= i & -i) s += t[i];
    return s; }
  T sum(int l, int r) { return sum(r) - sum(l); }   // [l, r)
  // smallest i with a[0]+..+a[i] >= s (all a >= 0), n if none
  int lowerBound(T s) {
    int pos = 0;
    for (int pw = 1 << __lg(n); pw; pw >>= 1)
      if (pos + pw <= n && t[pos + pw] < s)
        pos += pw, s -= t[pos];
    return pos; }
```

```
};
// Inversions: go left->right, ans += i - sum(a[i] + 1),
// add(a[i], 1)  (compress values to 0..n-1 first).
// Range add + point query: add(l, v), add(r, -v);
// then a[i] = sum(i + 1).
```

Segment tree (point update)

2-ds/segtree.cpp

How: pass `ID` and `f`: sum (0LL), min (`LLONG_MAX`), max (`LLONG_MIN`), gcd (0), or a struct (e.g. max subarray sum: {sum, pref, suf, best}). `f` must be associative, need not be commutative. Point add: `set(i, get(i) + v)`.

```
// Point update, range query [l, r) for any associative f
// (need not be commutative). T is deduced from ID. O(log n)
//   Seg mx(n, LLONG_MIN,
//          [](ll a, ll b) { return max(a, b); });
//   Seg sm(n, 0LL, [](ll a, ll b) { return a + b; });  // 0LL!
template <class T, class F> struct Seg {
  int n; T ID; F f; vector<T> t;
  Seg(int n, T ID, F f) : n(n), ID(ID), f(f), t(2 * n, ID) {}
  void set(int i, T v) {
    for (t[i += n] = v; i >>= 1;)
      t[i] = f(t[2 * i], t[2 * i + 1]); }
  T get(int i) { return t[i + n]; }
  T query(int l, int r) {
    T a = ID, b = ID;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) a = f(a, t[l++]);
      if (r & 1) b = f(t[--r], b);
    }
    return f(a, b); }
};
```

Lazy segment tree (generic)

2-ds/lazyseg.cpp

How: fill in 6 things: node `S` with `op` and neutral `e`; tag `F` with `mp(f, s)` (apply tag to a node; keep `len` in `S` if sums need it), `comp(f, g)` (newer `f` after older `g`) and identity `id`. Check: `mp(comp(f,g), s) = mp(f, mp(g, s))`. Ready instances: add/min/sum below, chmin/sum (beats) next. **Pitfalls:** `e` must be neutral (min: `LLONG_MAX`).

```
// GENERIC lazy segment tree (range update, range query).
//  S: node value; op(S, S) associative; e = neutral S
//  F: tag; mp(f, s) = s after applying f; comp(f, g) = "f
//     after g"; id = identity tag.
// BEATS: give S a field `bool fail`; mp sets s.fail = true
// when it cannot update the node exactly -> the tree pushes
// and recurses (mp on a leaf must never fail).
// findFirst(l, pred): first i >= l with !pred(op(a[l..i])),
// n if none; pred must be monotone and pred(e) = true.
template <class T, class = void>
struct HasFail : false_type {};
template <class T>
struct HasFail<T, void_t<decltype(declval<T>().fail)>>
    : true_type {};
template <class S, class F, class Op, class Mp, class Comp>
struct LazySeg {
  int n; S e; F id; Op op; Mp mp; Comp comp;
  vector<S> t; vector<F> lz;
  LazySeg(const vector<S>& a, S e, F id, Op op, Mp mp,
          Comp comp)
    : n(sz(a)), e(e), id(id), op(op), mp(mp), comp(comp),
      t(4 * n + 4, e), lz(4 * n + 4, id) {
    if (n) build(1, 0, n, a); }
  void build(int x, int l, int r, const vector<S>& a) {
    if (r - l == 1) { t[x] = a[l]; return; }
    int m = (l + r) / 2;
    build(2 * x, l, m, a); build(2 * x + 1, m, r, a);
    t[x] = op(t[2 * x], t[2 * x + 1]); }
  void put(int x, F f) {
    t[x] = mp(f, t[x]); lz[x] = comp(f, lz[x]); }
  void push(int x) {
    put(2 * x, lz[x]); put(2 * x + 1, lz[x]); lz[x] = id; }
  void update(int ql, int qr, F f, int x, int l, int r) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) {
      S s = mp(f, t[x]); bool bad = false;
      if constexpr (HasFail<S>::value) bad = s.fail;
      if (!bad) { t[x] = s; lz[x] = comp(f, lz[x]); return; }
    }
    push(x); int m = (l + r) / 2;
    update(ql, qr, f, 2 * x, l, m);
    update(ql, qr, f, 2 * x + 1, m, r);
    t[x] = op(t[2 * x], t[2 * x + 1]); }
  S query(int ql, int qr, int x, int l, int r) {
    if (qr <= l || r <= ql) return e;
    if (ql <= l && r <= qr) return t[x];
    push(x); int m = (l + r) / 2;
    return op(query(ql, qr, 2 * x, l, m),
              query(ql, qr, 2 * x + 1, m, r)); }
  template <class G>
  int find(int ql, G& pred, S& acc, int x, int l, int r) {
    if (r <= ql) return n;
    if (ql <= l && pred(op(acc, t[x]))) {
      acc = op(acc, t[x]); return n; }
    if (r - l == 1) return l;
    push(x); int m = (l + r) / 2;
    int i = find(ql, pred, acc, 2 * x, l, m);
    return i < n ? i : find(ql, pred, acc, 2 * x + 1, m, r); }
  void update(int l, int r, F f) { update(l, r, f, 1, 0, n); }
  S query(int l, int r) { return query(l, r, 1, 0, n); }
  template <class G> int findFirst(int l, G pred) {
    S acc = e; return find(l, pred, acc, 1, 0, n); }
};
// EXAMPLE: range add, range min + sum (len for the sum).
struct SN { ll mn, sm; int len; };
auto makeAddMinSum(const vector<ll>& a) {
  vector<SN> v;
  for (ll x : a) v.push_back({x, x, 1});
  return LazySeg(v, SN{LLONG_MAX, 0, 0}, 0LL,
    [](SN a, SN b) {
      return SN{min(a.mn, b.mn), a.sm + b.sm,
                a.len + b.len}; },
    [](ll f, SN s) {
      return SN{s.mn + f, s.sm + f * s.len, s.len}; },
    [](ll f, ll g) { return f + g; }); }
// first i >= l with a[i] < val:
//   st.findFirst(l, [&](SN s) { return s.mn >= val; });
// Range ASSIGN: F = {bool has; ll v},
// comp(f, g) = f.has ? f : g; assign + add: F = {has, v, add}.
```

Sparse table

2-ds/sparse.cpp

When: static array, many min/max/gcd queries (idempotent operations only; for sums use prefix sums).

```
// Static range query [l, r) in O(1) after O(n log n) build,
// for IDEMPOTENT f (min, max, gcd, and, or).
//   Sparse sp(a, [](int x, int y) { return min(x, y); });
template <class T, class F> struct Sparse {
  vector<vector<T>> t; F f;
  Sparse(const vector<T>& a, F f) : t{a}, f(f) {
    for (int k = 1; (1 << k) <= sz(a); k++) {
      t.emplace_back(sz(a) - (1 << k) + 1);
      for (int i = 0; i < sz(t[k]); i++)
        t[k][i] = f(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    }
  }
  T query(int l, int r) {            // requires l < r
    int k = __lg(r - l);
    return f(t[k][l], t[k][r - (1 << k)]); }
};
```

Monotonic stack, sliding window

2-ds/monostack.cpp

```
// For each i: L[i] = nearest j < i with a[j] < a[i] (or -1),
// R[i] = nearest j > i with a[j] <= a[i] (or n). O(n)
// ("<" / "<=" pair avoids double counting equal minima:
// sum of mins of all subarrays = sum a[i]*(i-L[i])*(R[i]-i))
void nearestSmaller(const vector<ll>& a, vi& L, vi& R) {
  int n = sz(a); L.assign(n, -1); R.assign(n, n);
  vi st;
  for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.back()] >= a[i]) {
      R[st.back()] = i; st.pop_back(); }
    if (!st.empty()) L[i] = st.back();
    st.push_back(i);
  }
}
// Sliding window max: res[i - k + 1] = max of a[i-k+1 .. i].
// Deque of indices with decreasing values. O(n). Min: flip <=.
vector<ll> windowMax(const vector<ll>& a, int k) {
  deque<int> q; vector<ll> res;
  for (int i = 0; i < sz(a); i++) {
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i);
    if (q.front() <= i - k) q.pop_front();
    if (i >= k - 1) res.push_back(a[q.front()]);
  }
  return res; }
```

Ordered set, hash map

2-ds/pbds.cpp

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
template <class T>
using oset = tree<T, null_type, less<T>, rb_tree_tag,
                  tree_order_statistics_node_update>;
// s.find_by_order(k): iterator to k-th smallest (0-based)
```

```
// s.order_of_key(x):  number of elements < x
// multiset: store pair<T, int> {value, unique id}

// Anti-hack hash for unordered_map / gp_hash_table
struct SafeHash {
  size_t operator()(uint64_t x) const {
    static const uint64_t R =
      chrono::steady_clock::now().time_since_epoch().count();
    x += R + 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31); }
};
// unordered_map<ll, int, SafeHash> m; m.reserve(1 << 20);
// gp_hash_table<ll, int, SafeHash> h;   (faster)
```

Li Chao tree (lines, min)

2-ds/lichao.cpp

When: DP of the form $dp_i = \min_j(k_j x_i + b_j)$ (convex hull trick) with lines and queries in any order. **How:** add the line of j after computing dp_j; query at x_i. Range of x is the integer domain $[lo, hi)$; nodes are created on demand, so $\pm 10^9$ is fine. **Pitfalls:** $k \cdot x + b$ must fit in `ll`.

```
// Li Chao tree: add line y = k x + b, query min at integer x
// in [lo, hi). O(log C) each. For max: negate k, b, answer.
struct LiChao {
  struct Line { ll k, b; ll operator()(ll x) const {
    return k * x + b; } };
  ll lo, hi; vector<Line> t; vi L, R;
  LiChao(ll lo, ll hi) : lo(lo), hi(hi) { node(); }
  int node() { t.push_back({0, LLONG_MAX}); L.push_back(-1);
    R.push_back(-1); return sz(t) - 1; }
  void add(Line nw) { add(nw, 0, lo, hi); }
  void add(Line nw, int x, ll l, ll r) {
    ll m = l + (r - l) / 2;
    bool lef = nw(l) < t[x](l), mid = nw(m) < t[x](m);
    if (mid) swap(t[x], nw);
    if (r - l == 1 || nw.b == LLONG_MAX) return;
    if (lef != mid) {
      if (L[x] < 0) { int c = node(); L[x] = c; }
      add(nw, L[x], l, m);
    } else {
      if (R[x] < 0) { int c = node(); R[x] = c; }
      add(nw, R[x], m, r);
    } }
  ll query(ll q) {
    ll res = LLONG_MAX; int x = 0; ll l = lo, r = hi;
    while (x >= 0) {
      res = min(res, t[x](q));
      ll m = l + (r - l) / 2;
      if (q < m) x = L[x], r = m; else x = R[x], l = m;
    }
    return res; }
};
```

Implicit treap

2-ds/treap.cpp

When: an array with insert / erase / reverse / cut-and-paste at arbitrary positions plus range aggregates. **How:** every operation = `split` into pieces, modify a piece (set a tag), `merge` back. More tags: handle them in `push` like `Rev`, recompute in `pull`.

```
// Implicit treap (array with split/merge): insert, erase,
// reverse, range sum at any position. O(log n) expected.
const int MX = 1e6 + 5;            // node 0 = empty
int L[MX], R[MX], S[MX], cnt; unsigned Pri[MX];
ll V[MX], Sum[MX]; bool Rev[MX];
mt19937 trng(123);
int node(ll v) {
  ++cnt; Pri[cnt] = trng(); S[cnt] = 1; V[cnt] = Sum[cnt] = v;
  return cnt; }
void push(int x) {
  if (x && Rev[x]) {
    swap(L[x], R[x]); Rev[L[x]] ^= 1; Rev[R[x]] ^= 1;
    Rev[x] = 0; } }
void pull(int x) {
  S[x] = 1 + S[L[x]] + S[R[x]];
  Sum[x] = V[x] + Sum[L[x]] + Sum[R[x]]; }
// a = first k elements of t, b = the rest
void split(int t, int k, int& a, int& b) {
  if (!t) { a = b = 0; return; }
  push(t);
  if (S[L[t]] < k)
    split(R[t], k - S[L[t]] - 1, R[t], b), a = t;
  else split(L[t], k, a, L[t]), b = t;
  pull(t); }
int merge(int a, int b) {
  if (!a || !b) return a | b;
  if (Pri[a] > Pri[b]) { push(a); R[a] = merge(R[a], b);
                         pull(a); return a; }
  push(b); L[b] = merge(a, L[b]); pull(b); return b; }
// reverse [l, r): split(root, r, a, c); split(a, l, a, b);
// Rev[b] ^= 1; root = merge(merge(a, b), c);
// insert v at pos i: split(root, i, a, b);
// root = merge(merge(a, node(v)), b);
```

DSU with rollback

2-ds/dsu_rollback.cpp

When: offline dynamic connectivity (edges added and removed): each edge lives on a time interval; put it into the $O(\log q)$ segment-tree nodes over time covering it; DFS that tree: unite the node's edges, recurse, `rollback`. Also bipartiteness over time (DSU with parity).

```
// DSU with rollback (union by size, NO path compression):
// O(log n) per find. t = time(); ...; rollback(t) undoes
// all unions made after t.
struct RDSU {
  vi p; vector<pii> st; int comps;      // p[root] = -size
  RDSU(int n) : p(n, -1), comps(n) {}
  int find(int x) { while (p[x] >= 0) x = p[x]; return x; }
  bool unite(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (p[a] > p[b]) swap(a, b);
    st.push_back({b, p[b]}); p[a] += p[b]; p[b] = a; comps--;
    return true; }
  int time() { return sz(st); }
  void rollback(int t) {
    while (sz(st) > t) {
      auto [b, pb] = st.back(); st.pop_back();
      p[p[b]] -= pb; p[b] = pb; comps++;
    } }
};
```

Mo's algorithm

2-ds/mo.cpp

When: static array, many range queries whose answer is easy to update when one element enters/leaves the window (number of distinct values, pairs of equal values, mode with counts). Offline only. $n = q = 2 \cdot 10^5$: about 10^8 add/rem calls.

```
// Mo's algorithm: offline queries [l, r) on a static array.
// add(i) / rem(i) change the current window, get() = answer.
// O((n + q) sqrt n) calls of add/rem: keep them O(1).
template <class Add, class Rem, class Get>
vector<ll> mo(int n, const vector<pii>& qs, Add add, Rem rem,
              Get get) {
  int q = sz(qs), B = max(1, int(n / sqrt(max(q, 1))));
  vi ord(q); iota(all(ord), 0);
  sort(all(ord), [&](int a, int b) {
    int x = qs[a].first / B, y = qs[b].first / B;
    if (x != y) return x < y;
    return x & 1 ? qs[a].second > qs[b].second
                 : qs[a].second < qs[b].second; });
  vector<ll> res(q); int l = 0, r = 0;   // window [l, r)
  for (int i : ord) {
    auto [ql, qr] = qs[i];
    while (r < qr) add(r++);
    while (l > ql) add(--l);
    while (r > qr) rem(--r);
    while (l < ql) rem(l++);
    res[i] = get();
  }
  return res; }
```

Mo's algorithm with rollback

2-ds/morollback.cpp

When: the window answer can be updated by *adding* but not by removing (max, max xor via trie, DSU). Short queries (inside one block) are brute-forced.

```
// Mo with rollback: when remove is impossible (max, max xor
// with a trie, DSU) but add + undo is easy. Queries [l, r).
// snap() returns a checkpoint (e.g. history size), roll(t)
// undoes back to it; get() reads the answer (keep it in the
// undoable state). O((n + q) sqrt n) adds.
template <class Add, class Get, class Snap, class Roll>
vector<ll> moRollback(int n, const vector<pii>& qs, Add add,
                      Get get, Snap snap, Roll roll) {
  int q = sz(qs), B = max(1, (int)sqrt(n));
  vi ord(q); iota(all(ord), 0);
  sort(all(ord), [&](int a, int b) {
    if (qs[a].first / B != qs[b].first / B)
      return qs[a].first / B < qs[b].first / B;
    return qs[a].second < qs[b].second; });
  vector<ll> res(q); auto base = snap();
  int cur = -1, r = 0;
  for (int i : ord) {
    auto [ql, qr] = qs[i];
    int b = ql / B, bend = min(n, (b + 1) * B);
    if (qr <= bend) {                  // short: brute force
      roll(base); cur = -1;
```

```
      for (int j = ql; j < qr; j++) add(j);
      res[i] = get(); roll(base); continue;
    }
    if (b != cur) { roll(base); cur = b; r = bend; }
    while (r < qr) add(r++);
    auto t = snap();
    for (int j = bend - 1; j >= ql; j--) add(j);
    res[i] = get(); roll(t);
  }
  return res; }
```

Persistent segment tree

2-ds/persistent.cpp

When: queries on old versions or on "prefix r minus prefix l": k-th smallest in a range, count of values $\le x$ in a range (sum `Cnt` of left children while descending).

```
// Persistent segment tree (counts over values [0, V)).
// Every padd makes a new version with O(log V) new nodes;
// node 0 = empty tree (version 0). Memory: n * (log2 V + 2).
// k-th smallest of a[l, r) (k 0-based, values compressed):
//   root[0] = 0; root[i + 1] = padd(root[i], 0, V, a[i]);
//   kth(root[l], root[r], 0, V, k)
const int PM = 2e5 * 20;
int Lc[PM], Rc[PM], Cnt[PM], pn = 1;
int padd(int t, int l, int r, int pos) {
  int x = pn++;
  Lc[x] = Lc[t]; Rc[x] = Rc[t]; Cnt[x] = Cnt[t] + 1;
  if (r - l > 1) {
    int m = (l + r) / 2;
    if (pos < m) Lc[x] = padd(Lc[t], l, m, pos);
    else Rc[x] = padd(Rc[t], m, r, pos);
  }
  return x; }
int kth(int a, int b, int l, int r, int k) {   // tree b - a
  if (r - l == 1) return l;
  int m = (l + r) / 2, c = Cnt[Lc[b]] - Cnt[Lc[a]];
  return k < c ? kth(Lc[a], Lc[b], l, m, k)
               : kth(Rc[a], Rc[b], m, r, k - c); }
```

Segment tree beats (chmin, sum)

2-ds/beats.cpp

When: range "$a_i = \min(a_i, x)$" (or max via negation) together with sums. **How:** the tag applies exactly only if x is above the second maximum; otherwise `mp` sets `fail` and the generic tree recurses (amortised by the number of distinct values). Same pattern for other beats operations.

```
// Segment tree beats as an instance of LazySeg (lazyseg.cpp):
// range chmin a_i = min(a_i, x), range sum and max.
// Amortised O((n + q) log n). Chmax: store -a, chmin with -x.
// Node: max, strict second max, count of max, sum, fail.
struct BN { ll mx, smx, cnt, sum; bool fail = false; };
BN mergeBN(BN a, BN b) {
  if (a.mx < b.mx) swap(a, b);
  if (a.mx == b.mx)
    return {a.mx, max(a.smx, b.smx), a.cnt + b.cnt,
            a.sum + b.sum};
  return {a.mx, max(a.smx, b.mx), a.cnt, a.sum + b.sum}; }
auto makeBeats(const vector<ll>& a) {
  vector<BN> v;
  for (ll x : a) v.push_back({x, LLONG_MIN, 1, x});
  return LazySeg(v, BN{LLONG_MIN, LLONG_MIN, 0, 0}, LLONG_MAX,
    mergeBN,
    [](ll f, BN s) {                  // chmin with f
      if (f >= s.mx) return s;
      if (f > s.smx) { s.sum -= (s.mx - f) * s.cnt; s.mx = f; }
      else s.fail = true;         // 2nd max too: descend
      return s; },
    [](ll f, ll g) { return min(f, g); }); }
// st.update(l, r, x) = chmin; st.query(l, r).sum / .mx
```

Records segment tree

2-ds/records.cpp

When: "number of visible buildings / prefix-maximum records" under point updates.

```
// "Visible buildings": point update; count of records (i with
// a[i] > max of everything before it in the range) on [l, r).
// res[x] = records of the right child given the left's max.
// calc is O(log n), so update/query O(log^2 n). For the SUM of
// record values store sums in res the same way.
struct Records {
  int n; vector<ll> mx; vi res;
  Records(int n) : n(n), mx(4 * n, LLONG_MIN), res(4 * n) {}
  int calc(int x, int l, int r, ll m) {  // records after max m
    if (mx[x] <= m) return 0;
    if (r - l == 1) return 1;
    int md = (l + r) / 2;
    if (mx[2 * x] <= m) return calc(2 * x + 1, md, r, m);
    return calc(2 * x, l, md, m) + res[x]; }
  void set(int i, ll v, int x, int l, int r) {
    if (r - l == 1) { mx[x] = v; return; }
    int md = (l + r) / 2;
    if (i < md) set(i, v, 2 * x, l, md);
    else set(i, v, 2 * x + 1, md, r);
    mx[x] = max(mx[2 * x], mx[2 * x + 1]);
    res[x] = calc(2 * x + 1, md, r, mx[2 * x]); }
  int query(int ql, int qr, ll& m, int x, int l, int r) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) {
      int c = calc(x, l, r, m); m = max(m, mx[x]); return c; }
    int md = (l + r) / 2;
    int a = query(ql, qr, m, 2 * x, l, md);
    return a + query(ql, qr, m, 2 * x + 1, md, r); }
  void set(int i, ll v) { set(i, v, 1, 0, n); }
  int query(int l, int r) {
    ll m = LLONG_MIN; return query(l, r, m, 1, 0, n); }
};
```

Dynamic segment tree

2-ds/dynseg.cpp

When: indices up to 10^{18} online (no compression possible), e.g. composing piecewise-linear functions, "first position where the prefix sum reaches t".

```
// Dynamic segment tree over [0, N), N up to 2^62: nodes made
// on demand (about 62 per update). Point add, range sum,
// lowerBound = smallest i with a[0] + .. + a[i] >= t (a >= 0).
// Node 0 = null (sum 0), node 1 = root.
struct DynSeg {
  ll N; vector<array<int, 2>> ch; vector<ll> s;
  DynSeg(ll N) : N(N), ch(2), s(2) {}
  int child(int x, int d) {
    if (!ch[x][d]) {
      ch[x][d] = sz(s); ch.push_back({0, 0}); s.push_back(0); }
    return ch[x][d]; }
  void add(ll i, ll v) {
    int x = 1; ll l = 0, r = N; s[x] += v;
    while (r - l > 1) {
      ll m = l + (r - l) / 2; int d = i >= m;
      if (d) l = m; else r = m;
      x = child(x, d); s[x] += v;
    } }
  ll sum(ll ql, ll qr, int x, ll l, ll r) {
    if (!x || qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return s[x];
    ll m = l + (r - l) / 2;
    return sum(ql, qr, ch[x][0], l, m) +
           sum(ql, qr, ch[x][1], m, r); }
  ll sum(ll l, ll r) { return sum(l, r, 1, 0, N); }
  ll lowerBound(ll t) {                    // N if none
    if (s[1] < t) return N;
    int x = 1; ll l = 0, r = N;
    while (r - l > 1) {
      ll m = l + (r - l) / 2; int L = ch[x][0];
      if (s[L] >= t) x = L, r = m;
      else t -= s[L], x = ch[x][1], l = m;
    }
    return l; }
};
```

2D Fenwick (generic prefix)

2-ds/bit2d.cpp

When: DP over pairs/points where the best predecessor lies in a dominated quadrant (sort by the first key, query the other two).

```
// 2D Fenwick: point "combine", query over [0, x) x [0, y).
// f commutative + associative: max with raise-only updates,
// or sum (+) with additions. O(log n log m), memory n*m.
//   BIT2D b(n, m, LLONG_MIN,
//           [](ll a, ll c) { return max(a, c); });
// Typical: DP over pairs sorted by one key, best over a
// dominated quadrant of the other two.
template <class T, class F> struct BIT2D {
  int n, m; T ID; F f; vector<vector<T>> t;
  BIT2D(int n, int m, T ID, F f)
    : n(n), m(m), ID(ID), f(f),
      t(n + 1, vector<T>(m + 1, ID)) {}
  void upd(int x, int y, T v) {
    for (int i = x + 1; i <= n; i += i & -i)
      for (int j = y + 1; j <= m; j += j & -j)
        t[i][j] = f(t[i][j], v); }
  T query(int x, int y) {
    T r = ID;
    for (int i = x; i > 0; i -= i & -i)
      for (int j = y; j > 0; j -= j & -j) r = f(r, t[i][j]);
    return r; }
};
```

CDQ divide and conquer

2-ds/cdq.cpp

When: offline counting of dominated points in 3 dimensions, "count pairs with $a_j \le a_i$, $b_j \le b_i$, $c_j \le c_i$", or offline queries over time with a Fenwick tree.

```
// CDQ divide & conquer: offline 3D dominance. For each point i
// count j != i with a_j <= a_i, b_j <= b_i, c_j <= c_i.
// Sort by (a, b, c), merge equal points (weight w).
// solve(l, r):
// recurse on halves; count (left j, right i) pairs by a sweep
// over b with a Fenwick tree on c; merge halves by b.
// O(n log^2 n). Needs fenwick.cpp; c compressed to [0, C).
// Same scheme: offline queries over time (left = updates,
// right = queries), 4D = CDQ inside CDQ.
vi dominance(const vector<array<int, 3>>& pts, int C) {
  int n = sz(pts); vi idx(n), grp(n);
  iota(all(idx), 0);
  sort(all(idx),
       [&](int x, int y) { return pts[x] < pts[y]; });
  vector<array<int, 3>> u; vi w;
  for (int i : idx) {
    if (u.empty() || u.back() != pts[i]) {
      u.push_back(pts[i]); w.push_back(0); }
    w.back()++; grp[i] = sz(u) - 1;
  }
  int m = sz(u); vi cnt(m), ord(m); iota(all(ord), 0);
  BIT bit(C);
  auto byB = [&](int x, int y) { return u[x][1] < u[y][1]; };
  auto solve = [&](auto&& self, int l, int r) -> void {
    if (r - l <= 1) return;
    int mid = (l + r) / 2;
    self(self, l, mid); self(self, mid, r);
    int j = l;
    for (int i = mid; i < r; i++) {
      for (; j < mid && u[ord[j]][1] <= u[ord[i]][1]; j++)
        bit.add(u[ord[j]][2], w[ord[j]]);
      cnt[ord[i]] += bit.sum(u[ord[i]][2] + 1);
    }
    for (int k = l; k < j; k++)          // undo the adds
      bit.add(u[ord[k]][2], -w[ord[k]]);
    inplace_merge(ord.begin() + l, ord.begin() + mid,
                  ord.begin() + r, byB);
  };
  solve(solve, 0, m);
  vi res(n);
  for (int i = 0; i < n; i++)
    res[i] = cnt[grp[i]] + w[grp[i]] - 1;
  return res; }
```

Ideas

- Operations like $a_i = \lfloor a_i/x \rfloor$, $\sqrt{a_i}$, $a_i \bmod x$, $\lfloor \log a_i \rfloor$ on ranges: store min and max; if $f(\min) = f(\max)$ on a node, it becomes a range *assign*, else recurse. Each value changes $O(\log)$ times $\Rightarrow$ amortised fast.
- Range assign tag: (has, value), assign overrides add; "find first index $\geq l$ with property" = descent (`findFirst`). Bit flips on ranges: tag = flip flag, node stores count of ones.
- Zobrist / XOR hashing: give each value a random 64-bit key; a multiset/set is the XOR (or sum) of keys; flipping bits $[l, r)$ changes the hash by $P_r \oplus P_l$ (prefix XORs of keys).
- Intervals in a segment tree: store each interval in its $O(\log n)$ canonical nodes; a point query (stabbing) walks the root–leaf path. DSU with "next unmerged" pointers skips already-processed elements: every element is merged/visited once.
- Sqrt on time: rebuild the static structure every $\sqrt{q}$ updates; answer queries from the structure + the pending updates.
- Range-sum of convolution coefficients ($\sum_p a_p a_{c-p}$ over $p \in [l, r]$): anti-diagonal prefix sums, no FFT.
- Pareto front (staircase) DP on trees: keep only non-dominated (w,h) pairs per subtree, merge two fronts with two pointers, prune by a bound.
- Offline: sort queries by right end + Fenwick/segment tree ("number of distinct values on $[l, r]$": keep only the last occurrence). Process in reverse to turn deletions into DSU unions.
- Sqrt decomposition: blocks of $\approx \sqrt{n}$; Mo's order for $[l, r)$ queries: sort by $(l/B,\ r)$, alternate r direction for odd blocks; $B \approx n/\sqrt{q}$.
- Small-to-large merging of sets: $O(n \log^2 n)$ total.
- Segment tree descent: first index with prefix $\geq x$ in $O(\log n)$ (walk down from the root).
- Persistent: copy the $O(\log n)$ nodes on the path on each update.

4 Graphs

Dijkstra, 0-1 BFS, Bellman–Ford 3-graph/shortest.cpp

```
// Dijkstra, w >= 0. g[u] = {(v, w)}. O(m log m)
using Adj = vector<vector<pair<int, ll>>>;
const ll INF = 4e18;
vector<ll> dijkstra(const Adj& g, int s) {
  vector<ll> d(sz(g), INF);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                 greater<>> pq;
  pq.push({d[s] = 0, s});
  while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != d[u]) continue;
    for (auto [v, w] : g[u])
      if (du + w < d[v]) pq.push({d[v] = du + w, v});
  }
  return d; }
// 0-1 BFS: weights 0/1, deque; w=0 -> push_front, else back.
vector<ll> bfs01(const Adj& g, int s) {
  vector<ll> d(sz(g), INF); deque<int> q{s}; d[s] = 0;
  while (!q.empty()) {
    int u = q.front(); q.pop_front();
    for (auto [v, w] : g[u]) if (d[u] + w < d[v]) {
      d[v] = d[u] + w;
      w ? q.push_back(v) : q.push_front(v); }
  }
  return d; }
// Bellman-Ford, any weights; edges (u, v, w). O(nm).
// Returns false if a negative cycle is reachable from s.
bool bellman(int n, vector<array<ll, 3>>& e, int s,
             vector<ll>& d) {
  d.assign(n, INF); d[s] = 0;
  for (int it = 0; it < n; it++) {
    bool ch = false;
    for (auto [u, v, w] : e)
      if (d[u] < INF && d[u] + w < d[v])
        d[v] = d[u] + w, ch = true;
    if (!ch) return true;
  }
  return false; }
// Floyd: for k: for i: for j:  (k is the OUTER loop!)
//   d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
// Negative cycle <=> some d[i][i] < 0.
```

Topological sort 3-graph/topo.cpp

```
// Kahn topological order of a DAG; size < n => has a cycle.
// For lexicographically smallest order use a min-heap.
vi topo(const vector<vi>& g) {
  int n = sz(g); vi indeg(n), order;
  for (auto& es : g) for (int v : es) indeg[v]++;
  for (int i = 0; i < n; i++)
    if (!indeg[i]) order.push_back(i);
  for (int i = 0; i < sz(order); i++)
    for (int v : g[order[i]])
      if (!--indeg[v]) order.push_back(v);
  return order; }
// Bipartite check: BFS coloring c[v] = c[u] ^ 1; conflict =>
// odd cycle. Grid BFS: dx[] = {1,-1,0,0}, dy[] = {0,0,1,-1}.
```

SCC (Tarjan) and 2-SAT 3-graph/scc.cpp

How: 2-SAT: write every constraint as clauses "a or b". "$a \Rightarrow b$" = `either(a, b)`; "$a = b$" = two clauses; "exactly one" = $(a \vee b) \wedge (\neg a \vee \neg b)$. SCC condensation is a DAG: DP on it in order $ncomp - 1 \to 0$ (comp 0 is a sink).

```
// Tarjan SCC. comp[v] in [0, ncomp); components are numbered
// in REVERSE topological order (comp 0 is a sink). O(n + m)
struct SCC {
  int n, timer = 0, ncomp = 0;
  vector<vi> g; vi tin, low, comp, st;
  SCC(int n) : n(n), g(n), tin(n, -1), low(n), comp(n, -1) {}
  void dfs(int u) {
    tin[u] = low[u] = timer++; st.push_back(u);
    for (int v : g[u]) {
      if (tin[v] < 0) dfs(v), low[u] = min(low[u], low[v]);
      else if (comp[v] < 0) low[u] = min(low[u], tin[v]);
    }
    if (low[u] == tin[u]) {
      int v;
      do { v = st.back(); st.pop_back(); comp[v] = ncomp; }
      while (v != u);
      ncomp++;
    } }
  void run() {
    for (int i = 0; i < n; i++) if (tin[i] < 0) dfs(i); }
};
// 2-SAT: variable x -> nodes 2x (x true), 2x+1 (x false).
struct TwoSat {
  int n; SCC s;
  TwoSat(int n) : n(n), s(2 * n) {}
  // clause (x == vx) OR (y == vy)
  void either(int x, bool vx, int y, bool vy) {
    int a = 2 * x + !vx, b = 2 * y + !vy;
    s.g[a ^ 1].push_back(b); s.g[b ^ 1].push_back(a); }
```

```
  void force(int x, bool v) { either(x, v, x, v); }
  bool solve(vector<bool>& val) {
    s.run(); val.assign(n, false);
    for (int i = 0; i < n; i++) {
      if (s.comp[2 * i] == s.comp[2 * i + 1]) return false;
      val[i] = s.comp[2 * i] < s.comp[2 * i + 1];
    }
    return true; }
};
```

Bridges and articulation points

3-graph/bridges.cpp

When: critical edges/vertices. Contract 2-edge-connected components (DFS without bridges) ⇒ the *bridge tree*; many problems become tree problems on it.

```
// Bridges and articulation points (multi-edges OK). O(n + m)
// 2-edge-connected comps: DFS/DSU over non-bridge edges.
struct Bridges {
  int timer = 0;
  vector<vector<pii>> g;              // (to, edge id)
  vi tin, low; vector<bool> isBridge, isCut;
  Bridges(int n, int m)
    : g(n), tin(n, -1), low(n), isBridge(m), isCut(n) {}
  void addEdge(int u, int v, int id) {
    g[u].push_back({v, id}); g[v].push_back({u, id}); }
  void dfs(int u, int pe) {
    tin[u] = low[u] = timer++; int kids = 0;
    for (auto [v, id] : g[u]) {
      if (id == pe) continue;
      if (tin[v] >= 0) {
        low[u] = min(low[u], tin[v]); continue; }
      dfs(v, id); kids++;
      low[u] = min(low[u], low[v]);
      if (low[v] > tin[u]) isBridge[id] = true;
      if (pe >= 0 && low[v] >= tin[u]) isCut[u] = true;
    }
    if (pe < 0 && kids > 1) isCut[u] = true; }
  void run() {
    for (int i = 0; i < sz(g); i++)
      if (tin[i] < 0) dfs(i, -1); }
};
```

LCA (binary lifting)

3-graph/lca.cpp

When: distances/path sums on a static tree: $f(u,v) = p_u + p_v - 2p_{lca}$ for root-prefix values p. `kth` gives the k-th vertex on a path.

```
// LCA by binary lifting on a tree (adjacency list).
// O(n log n) build, O(log n) query. Iterative (no deep DFS).
struct LCA {
  int L; vector<vi> up; vi dep, order;
  LCA(const vector<vi>& g, int root = 0)
    : L(__lg(max(1, sz(g))) + 1),
      up(L, vi(sz(g), root)), dep(sz(g)) {
    order = {root};                 // BFS order
    vector<bool> seen(sz(g)); seen[root] = true;
    for (int i = 0; i < sz(order); i++)
      for (int v : g[order[i]]) if (!seen[v]) {
        seen[v] = true; up[0][v] = order[i];
        dep[v] = dep[order[i]] + 1; order.push_back(v); }
    for (int k = 1; k < L; k++)
      for (int v = 0; v < sz(g); v++)
        up[k][v] = up[k - 1][up[k - 1][v]];
  }
  int kth(int v, int k) {           // k-th ancestor (k <= dep)
    for (int i = 0; i < L; i++) if (k >> i & 1) v = up[i][v];
    return v; }
  int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a, b);
    a = kth(a, dep[a] - dep[b]);
    if (a == b) return a;
    for (int k = L - 1; k >= 0; k--)
      if (up[k][a] != up[k][b]) a = up[k][a], b = up[k][b];
    return up[0][a]; }
  int dist(int a, int b) {
    return dep[a] + dep[b] - 2 * dep[lca(a, b)]; }
};
// Tree DP bottom-up without recursion: for v in reversed
// order: combine v into up[0][v].  Subtree as a range:
// Euler tour tin/tout; v is an ancestor of u
// <=> tin[v] <= tin[u] < tout[v].
```

Heavy-light decomposition

3-graph/hld.cpp

When: path update + path query (sum/max/assign) on a tree that changes values: $O(\log^2 n)$ per query with a segment tree on `pos`. Subtree query = one range. `pos` is a DFS preorder. **Pitfalls:** recursion depth n.

```
// Heavy-light decomposition. Path u-v = O(log n) ranges of
// positions [l, r); put values at pos[v] in a segment tree.
// Subtree of v = [pos[v], pos[v] + size[v]).
// Values on EDGES: store edge (par v, v) at pos[v] and skip
// the LCA: in the last step use pos[a] + 1.
struct HLD {
  vi par, dep, heavy, head, pos, size; int timer = 0;
  HLD(const vector<vi>& g, int root = 0) : par(sz(g), -1),
      dep(sz(g)), heavy(sz(g), -1), head(sz(g)), pos(sz(g)),
      size(sz(g), 1) {
    dfs(g, root); decompose(g, root, root); }
  void dfs(const vector<vi>& g, int u) {
    for (int v : g[u]) if (v != par[u]) {
      par[v] = u; dep[v] = dep[u] + 1; dfs(g, v);
      size[u] += size[v];
      if (heavy[u] < 0 || size[v] > size[heavy[u]])
        heavy[u] = v;
    } }
  void decompose(const vector<vi>& g, int u, int h) {
    head[u] = h; pos[u] = timer++;
    if (heavy[u] >= 0) decompose(g, heavy[u], h);
    for (int v : g[u])
      if (v != par[u] && v != heavy[u]) decompose(g, v, v); }
  template <class F> void path(int a, int b, F f) {
    for (; head[a] != head[b]; b = par[head[b]]) {
      if (dep[head[a]] > dep[head[b]]) swap(a, b);
      f(pos[head[b]], pos[b] + 1);
    }
    if (dep[a] > dep[b]) swap(a, b);
    f(pos[a], pos[b] + 1); }    // a = LCA here
  int lca(int a, int b) {
    for (; head[a] != head[b]; b = par[head[b]])
      if (dep[head[a]] > dep[head[b]]) swap(a, b);
    return dep[a] < dep[b] ? a : b; }
};
// usage: h.path(u, v, [&](int l, int r) {
//          ans += seg.query(l, r); });
```

Centroid decomposition

3-graph/centroid.cpp

When: counting/optimising over *all paths* (number of paths of length $\le K$, with sum $= X$), or "nearest marked vertex" queries. **How:** every path is handled at the highest centroid on it; at centroid c, DFS each child subtree, combine with data from previous subtrees (avoid pairs inside one subtree). Nearest marked: `best[c]` for every centroid ancestor c of v; answer $= \min_c$ `best[c]` $+ dist(v,c)$; depth $\le \log n$.

```
// Centroid decomposition: every path passes through the
// centroid that is highest in the centroid tree. O(n log n)
// times the work per level.
vector<vi> g; vi sub, cpar; vector<bool> removed;
int calcSize(int u, int p) {
  sub[u] = 1;
  for (int v : g[u]) if (v != p && !removed[v])
    sub[u] += calcSize(v, u);
  return sub[u]; }
int findCentroid(int u, int p, int n) {
  for (int v : g[u])
    if (v != p && !removed[v] && 2 * sub[v] > n)
      return findCentroid(v, u, n);
  return u; }
void build(int u, int p = -1) {   // sub, cpar, removed sized n
  int c = findCentroid(u, -1, calcSize(u, -1));
  cpar[c] = p; removed[c] = true;
  // ... count paths through c: DFS into each child subtree
  // of c (only !removed vertices), combine with earlier ones
  for (int v : g[c]) if (!removed[v]) build(v, c);
}
```

Virtual tree

3-graph/virtualtree.cpp

When: many queries, each on a small set of k vertices with $\sum k$ bounded: build a tree of size $< 2k$ and run the DP only there.

```
// Virtual (auxiliary) tree of key vertices ks: the keys plus
// their pairwise LCAs, O(k log n). tin = DFS preorder (e.g.
// HLD::pos), lca = HLD::lca. Returns its vertices in preorder
// (first = root); vpar[u] = virtual parent (-1 for the root).
// Edge (vpar[u], u) stands for a path of dep[u]-dep[vpar[u]].
template <class F>
vi virtualTree(vi ks, const vi& tin, F lca, vi& vpar) {
  auto cmp = [&](int a, int b) { return tin[a] < tin[b]; };
  sort(all(ks), cmp);
  for (int i = 0, k = sz(ks); i + 1 < k; i++)
    ks.push_back(lca(ks[i], ks[i + 1]));
  sort(all(ks), cmp); ks.erase(unique(all(ks)), ks.end());
  for (int i = 0; i < sz(ks); i++)
    vpar[ks[i]] = i ? lca(ks[i - 1], ks[i]) : -1;
  return ks; }
// Typical use: sum of k over all queries <= 2e5; run a DP on
// the virtual tree (children = reverse of vpar) per query.
```

Small-to-large (DSU on tree)

3-graph/sack.cpp

When: "for every subtree, something about the multiset of values in it". Same idea for merging heaps, maps of counts, etc.

```
// Small-to-large merging: always insert the smaller set into
// the larger one (swap is O(1)) -> every element moves
// O(log n) times. Example: # distinct colours in each subtree.
// Recursive: a path of 1e5 overflows an 8MB stack (see the
// checklist: bigger stack or iterative order).
vector<vi> g; vi col, ans;
set<int> dfs(int u, int p) {
  set<int> s{col[u]};
  for (int v : g[u]) if (v != p) {
    set<int> t = dfs(v, u);
    if (sz(t) > sz(s)) swap(s, t);
    s.insert(all(t));
  }
  ans[u] = sz(s);
  return s; }
```

Block-cut tree

3-graph/blockcut.cpp

When: cut vertices, vertex-biconnected components, **cactus** problems (each cycle = one block, listed in cycle order: prefix sums around it give both arc lengths). **How:** DP/LCA on the block-cut tree (round = vertex, square = block).

```
// Block-cut tree (vertex-biconnected components). Nodes 0..n-1
// are vertices, n.. are blocks; tree edges vertex - block.
// Cut vertices = vertices with >= 2 blocks. CACTUS: each cycle
// is one block and tree[b] lists it IN CYCLE ORDER, starting
// at the block's top vertex tree[b][0] (nearest to the root).
// Simple graph (multi-edges: skip the parent EDGE id instead).
struct BlockCut {
  int timer = 0; vector<vi> g, tree; vi tin, low, st;
  BlockCut(const vector<vi>& g_) : g(g_), tree(sz(g)),
      tin(sz(g), -1), low(sz(g)) {
    for (int v = 0; v < sz(g); v++)
      if (tin[v] < 0) dfs(v, -1); }
  void dfs(int u, int p) {
    tin[u] = low[u] = timer++; st.push_back(u);
    for (int v : g[u]) {
      if (v == p) continue;
      if (tin[v] >= 0) {
        low[u] = min(low[u], tin[v]); continue; }
      dfs(v, u); low[u] = min(low[u], low[v]);
      if (low[v] >= tin[u]) {            // u tops a new block
        int b = sz(tree);
        tree.push_back({u}); tree[u].push_back(b);
        int w;
        do {
          w = st.back(); st.pop_back();
          tree[b].push_back(w); tree[w].push_back(b);
        } while (w != v);
      }
    } }
};
```

Functional graph

3-graph/functional.cpp

When: every vertex has one outgoing edge (permutations, "next" pointers, teleports): cycles + in-trees, k-th successor, distance to cycle.

```
// Functional graph v -> f[v] (out-degree 1): each component is
// one cycle with in-trees. Peel indegree-0 vertices (Kahn);
// what remains are cycles. O(n), no recursion.
// order = peeled vertices, leaves first: tree DP towards the
// cycle in this order (e.g. h[f[v]] = max(h[f[v]], h[v] + 1));
// distance to the cycle: dist[v] = dist[f[v]] + 1, going
// through order REVERSED.
// Cycle problems: work on the cycle doubled (length 2L) with
// prefix sums / sliding windows. k-th successor: binary
// lifting on f.
struct Functional {
  vi f, onCyc, cycId, order; vector<vi> cycles;
  Functional(const vi& f_) : f(f_) {
    int n = sz(f); vi indeg(n);
    onCyc.assign(n, 1); cycId.assign(n, -1);
    for (int v : f) indeg[v]++;
    for (int v = 0; v < n; v++)
      if (!indeg[v]) order.push_back(v);
    for (int i = 0; i < sz(order); i++) {
      int v = order[i]; onCyc[v] = 0;
      if (!--indeg[f[v]]) order.push_back(f[v]);
    }
    for (int v = 0; v < n; v++) if (onCyc[v] && cycId[v] < 0) {
      cycles.emplace_back();
      for (int u = v; cycId[u] < 0; u = f[u])
        cycId[u] = sz(cycles) - 1, cycles.back().push_back(u);
    }
  }
};
```

Binary lifting with costs

3-graph/lifting.cpp

When: greedy "jump to the next allowed position" many times with a budget (time, fuel): build `nxt`, lift it, answer each query in $O(\log)$. Split tree paths at the LCA and run it from both ends.

```
// Binary lifting over any successor function nxt with costs
// c[v] (cost of one step from v): up[k][v] = 2^k-th successor,
// cs[k][v] = cost of those 2^k steps, capped at CAP.
// maxJumps(v, B) = {steps, end vertex}: most steps of total
// cost <= B (need B < CAP).
// Terminal vertex: nxt[t] = t with cost CAP (blocks jumps).
// Memory L * n: choose L = log2(max steps) + 1.
const ll CAP = 2e18;
struct Lift {
  int L; vector<vi> up; vector<vector<ll>> cs;
  Lift(const vi& nxt, const vector<ll>& c, int L)
    : L(L), up(L, nxt), cs(L, c) {
    for (int k = 1; k < L; k++)
      for (int v = 0; v < sz(nxt); v++) {
        int m = up[k - 1][v];
        up[k][v] = up[k - 1][m];
        cs[k][v] = min(CAP, cs[k - 1][v] + cs[k - 1][m]);
      } }
  pair<ll, int> maxJumps(int v, ll B) {
    ll t = 0;
    for (int k = L - 1; k >= 0; k--)
      if (cs[k][v] <= B)
        B -= cs[k][v], t += 1LL << k, v = up[k][v];
    return {t, v}; }
};
// Tree version: nxt = parent; "climb while the weight sum
// fits", or "first ancestor with value > x" (store max).
```

Link-cut tree

3-graph/linkcut.cpp

When: the forest changes online (link/cut) and you ask connectivity / root / path sums. Graph with cycles (pointer graphs): keep one "spare" edge per cycle outside the LCT.

```
// Link-cut tree: dynamic forest, O(log n) amortised. Nodes
// 1..n (0 = null). Unrooted: link(u, v) joins two trees,
// cut(u, v) removes the edge u-v, connected(u, v).
// Rooted forest (fixed roots, e.g. pointer graphs): only
// linkRooted / cutParent / findRoot; never call makeRoot.
// Path sums: add val/sum, recompute in a pull() after rotate
// and access (like the treap).
struct LCT {
  vector<array<int, 2>> ch; vi p; vector<bool> rev;
  LCT(int n) : ch(n + 1), p(n + 1), rev(n + 1) {}
  bool isRoot(int x) {        // root of its splay tree
    return ch[p[x]][0] != x && ch[p[x]][1] != x; }
  void push(int x) {
    if (!rev[x]) return;
    swap(ch[x][0], ch[x][1]);
    for (int c : ch[x]) if (c) rev[c] = !rev[c];
    rev[x] = false; }
  void rotate(int x) {
    int y = p[x], z = p[y], d = ch[y][1] == x;
    if (!isRoot(y)) ch[z][ch[z][1] == y] = x;
    p[x] = z; ch[y][d] = ch[x][!d];
    if (ch[x][!d]) p[ch[x][!d]] = y;
    ch[x][!d] = y; p[y] = x; }
  void splay(int x) {
    vi st{x};
    for (int y = x; !isRoot(y); y = p[y]) st.push_back(p[y]);
    for (int i = sz(st) - 1; i >= 0; i--) push(st[i]);
    while (!isRoot(x)) {
      int y = p[x], z = p[y];
      if (!isRoot(y))
        rotate((ch[y][1] == x) == (ch[z][1] == y) ? y : x);
      rotate(x);
    } }
  void access(int x) {
    for (int last = 0; x; last = x, x = p[x]) {
      splay(x); ch[x][1] = last; } }
  void makeRoot(int x) {
    access(x); splay(x); rev[x] = !rev[x]; }
  int findRoot(int x) {
    access(x); splay(x);
    while (push(x), ch[x][0]) x = ch[x][0];
    splay(x); return x; }
  bool connected(int u, int v) {
    return findRoot(u) == findRoot(v); }
  void link(int u, int v) { makeRoot(u); p[u] = v; }
  void cut(int u, int v) {              // edge u-v must exist
    makeRoot(u); access(v); splay(v); ch[v][0] = p[u] = 0; }
  void linkRooted(int u, int v) {       // u is a tree root
```

```
    access(u); splay(u); p[u] = v; }
  void cutParent(int u) {                  // u is not a root
    access(u); splay(u); p[ch[u][0]] = 0; ch[u][0] = 0; }
};
```

Euler path

3-graph/euler.cpp

```
// Euler path/cycle (Hierholzer). g[u] = {(v, edge id)};
// undirected: add the edge to both ends with the same id.
// Start: undirected -> a vertex of odd degree (if any);
// directed -> vertex with out - in = 1 (if any).
// Returns vertices of the path. First check the degrees (0/2
// odd; directed: out-in all 0 or one +1 and one -1); then
// size != m + 1 -> graph not connected -> no path.
vi euler(const vector<vector<pii>>& g, int m, int s) {
  vi it(sz(g)), path, st{s}; vector<bool> used(m);
  while (!st.empty()) {
    int u = st.back(); int& i = it[u];
    while (i < sz(g[u]) && used[g[u][i].second]) i++;
    if (i == sz(g[u])) { path.push_back(u); st.pop_back(); }
    else { used[g[u][i].second] = true;
           st.push_back(g[u][i].first); }
  }
  reverse(all(path));
  return path; }
```

Bipartite matching (Kuhn)

3-graph/matching.cpp

When: bipartite matching up to $\sim$ 5000 vertices per side; larger: Dinic ($O(E\sqrt{V})$). Shuffle the adjacency lists against bad cases.

```
// Bipartite matching (Kuhn), left 0..n-1 -> right 0..m-1.
// O(V E) worst, fast in practice. For big graphs use Dinic.
// Konig: min vertex cover = max matching;
// max independent set = n + m - matching.
struct Kuhn {
  int n, m; vector<vi> g; vi mt, vis; int stamp = 0;
  Kuhn(int n, int m) : n(n), m(m), g(n), mt(m, -1), vis(n) {}
  bool dfs(int u) {
    if (vis[u] == stamp) return false;
    vis[u] = stamp;
    for (int v : g[u])
      if (mt[v] < 0 || dfs(mt[v])) { mt[v] = u; return true; }
    return false; }
  int run() {
    int res = 0;
    for (int u = 0; u < n; u++) { stamp++; res += dfs(u); }
    return res; }       // mt[u] = left partner of right v
};
```

Max flow (Dinic)

3-graph/dinic.cpp

How: modelling: source $\to$ suppliers, consumers $\to$ sink, capacities = limits. Undirected edge: `addEdge(u, v, c, c)`. Vertex capacity: split v into $v_{in} \to v_{out}$. Min cut edges = saturated edges from `lvl`≥ 0 to `lvl`< 0. **Pitfalls:** `s != t`.

```
// Max flow (Dinic, multi-push). O(V^2 E); unit caps:
// O(E sqrt V). 1e5 edges in well under a second.
struct Dinic {
  struct E { int to; ll cap; };
  vector<E> e; vector<vi> g; vi lvl, it;
  Dinic(int n) : g(n), lvl(n), it(n) {}
  // rc = capacity of the reverse edge (undirected: rc = c)
  int addEdge(int u, int v, ll c, ll rc = 0) {
    g[u].push_back(sz(e)); e.push_back({v, c});
    g[v].push_back(sz(e)); e.push_back({u, rc});
    return sz(e) - 2; }     // flow on edge id = e[id ^ 1].cap
  bool bfs(int s, int t) {
    fill(all(lvl), -1); lvl[s] = 0;
    vi q{s};
    for (int i = 0; i < sz(q); i++)
      for (int id : g[q[i]])
        if (e[id].cap > 0 && lvl[e[id].to] < 0) {
          lvl[e[id].to] = lvl[q[i]] + 1;
          q.push_back(e[id].to); }
    return lvl[t] >= 0; }
  ll dfs(int u, int t, ll f) {     // pushes up to f from u
    if (u == t) return f;
    ll pushed = 0;
    for (int& i = it[u]; i < sz(g[u]); i++) {
      E& x = e[g[u][i]];
      if (x.cap <= 0 || lvl[x.to] != lvl[u] + 1) continue;
      ll d = dfs(x.to, t, min(f - pushed, x.cap));
      x.cap -= d; e[g[u][i] ^ 1].cap += d; pushed += d;
      if (pushed == f) return f;  // edge may have cap left
    }
    lvl[u] = -1;                  // dead end: prune
    return pushed; }
  ll maxflow(int s, int t) {
    ll flow = 0;
    while (bfs(s, t)) { fill(all(it), 0);
                        flow += dfs(s, t, LLONG_MAX); }
    return flow; }
};
// Min cut after maxflow(): S side = {v : lvl[v] >= 0}.
// Bipartite matching: s->L, L->R, R->t, all caps 1.
```

Min-cost flow

3-graph/mcmf.cpp

When: flow with costs: assignment, transport, "choose k disjoint paths of min total length". **How:** augments along cheapest paths, so with `lim = K` you get the cheapest K units. **Pitfalls:** negative cycles are not allowed (negative edges are).

```
// Min-cost max-flow (SPFA augmenting paths); negative costs
// OK if no negative cycle. O(flow * V * E) worst case.
struct MCMF {
  struct E { int to; ll cap, cost; };
  vector<E> e; vector<vi> g; vector<ll> d; vi pe;
  MCMF(int n) : g(n), d(n), pe(n) {}
  void addEdge(int u, int v, ll cap, ll cost) {
    g[u].push_back(sz(e)); e.push_back({v, cap, cost});
    g[v].push_back(sz(e)); e.push_back({u, 0, -cost}); }
  pair<ll, ll> run(int s, int t, ll lim = LLONG_MAX) {
    ll flow = 0, cost = 0; vector<bool> inq(sz(g));
    while (flow < lim) {
      fill(all(d), LLONG_MAX); d[s] = 0;
      deque<int> q{s};
      while (!q.empty()) {
        int u = q.front(); q.pop_front(); inq[u] = false;
        for (int id : g[u]) {
          auto& x = e[id];
          if (x.cap > 0 && d[u] + x.cost < d[x.to]) {
            d[x.to] = d[u] + x.cost; pe[x.to] = id;
            if (!inq[x.to])
              inq[x.to] = true, q.push_back(x.to);
          }
        }
      }
      if (d[t] == LLONG_MAX) break;
      ll f = lim - flow;
      for (int v = t; v != s; v = e[pe[v] ^ 1].to)
        f = min(f, e[pe[v]].cap);
      for (int v = t; v != s; v = e[pe[v] ^ 1].to)
        e[pe[v]].cap -= f, e[pe[v] ^ 1].cap += f;
      flow += f; cost += f * d[t];
    }
    return {flow, cost}; }
};
// Assignment problem n x n: s->row (1,0), row->col (1,c),
// col->t (1,0). Max cost: negate costs.
```

Hungarian algorithm

3-graph/hungarian.cpp

When: dense assignment problem, $n \leq 500$ ($n^2 m$). Simpler and faster than min-cost flow for this case.

```
// Hungarian algorithm: assign each of n rows a distinct
// column (n <= m) with min total cost a[i][j]. O(n^2 m).
// Returns {cost, column of each row}. Max: negate costs.
pair<ll, vi> hungarian(const vector<vector<ll>>& a) {
  int n = sz(a), m = sz(a[0]);
  vector<ll> u(n + 1), v(m + 1); vi p(m + 1), way(m + 1);
  for (int i = 1; i <= n; i++) {
    p[0] = i; int j0 = 0;
    vector<ll> minv(m + 1, LLONG_MAX);
    vector<bool> used(m + 1);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0; ll delta = LLONG_MAX;
      for (int j = 1; j <= m; j++) if (!used[j]) {
        ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < minv[j]) minv[j] = cur, way[j] = j0;
        if (minv[j] < delta) delta = minv[j], j1 = j;
      }
      for (int j = 0; j <= m; j++)
        if (used[j]) u[p[j]] += delta, v[j] -= delta;
        else minv[j] -= delta;
      j0 = j1;
    } while (p[j0]);
    do {                                  // augment along way[]
      int j1 = way[j0]; p[j0] = p[j1]; j0 = j1;
    } while (j0);
  }
  vi col(n);
  for (int j = 1; j <= m; j++) if (p[j]) col[p[j] - 1] = j - 1;
  return {-v[0], col}; }
```

Facts

- Tree: $n-1$ edges, connected. Diameter: farthest vertex a from any vertex, then farthest b from a. Center(s) on the diameter middle.
- Euler: undirected connected graph has an Euler cycle $\iff$ all degrees even; path $\iff$ 0 or 2 odd vertices. Directed: in = out everywhere (path: one vertex out−in= 1, one in−out= 1).
- Planar: $V-E+F=2$ (connected), $E \le 3V-6$. Cactus: #cycles $= E-V+1$.
- Bipartite $\iff$ no odd cycle. König: max matching = min vertex cover. Hall: perfect matching of $L \iff |N(S)| \ge |S|$ for all $S \subseteq L$. Min path cover in a DAG = $n-$ max matching (split v into v_{out}, v_{in}). Dilworth: min chains = max antichain.
- Max flow = min cut. Vertex capacities: split $v \to v_{in}, v_{out}$. Lower bounds l: add demand edges $S' \to v$, $u \to T'$. Max closure (project selection): $\sum$ positive − mincut (profit: $s \to v$, cost: $v \to t$, dependency $u \to v$ with ∞).
- MST: Kruskal (sort + DSU) or Prim $O(m \log n)$; minimum bottleneck path lies on the MST. Boruvka: $O(\log n)$ rounds, each vertex picks its cheapest outgoing edge.
- Shortest paths with k free edges / states: graph on (vertex, state). Dijkstra with potentials for negative edges after one Bellman–Ford.
- Functional graph (n vertices, out-degree 1): every component is a cycle with trees; use binary lifting for k-th successor.
- Tournament: has a Hamiltonian path; SCCs are linearly ordered.
- Orient undirected edges of a mixed graph to balance in/out degrees (Euler in mixed graphs, fewest added edges): orient arbitrarily, $b_v = out - in$; flow $s \to v$ (cap $b_v/2$, $b_v > 0$), $v \to t$ ($-b_v/2$), every undirected edge $u \to v$ cap 1 (flipping it). Extra edges needed $= \sum_{b_v>0} b_v - 2\cdot$maxflow (parities must match).
- Degree-constrained subgraph in a bipartite/directed graph (each vertex exactly/at most d_v edges): flow $s \to L$ (cap d), edges cap 1, $R \to t$ (cap d); feasible iff saturated. Few special edges: try all subsets of them.
- Arboricity (min #forests covering all edges, Nash-Williams): $\max_H \lceil |E_H|/(|V_H|-1) \rceil$; $K_{n,m}$: $\lceil nm/(n+m-1) \rceil$. A graph without isolated vertices splits into stars (cut a spanning forest bottom-up).
- Reachability in a DAG for all pairs: `bitset<N>` per vertex, OR along reversed topological order, $O(nm/64)$; low memory: process $64k$ target columns at a time.
- Long-path decomposition: the sum of the k longest root-to-leaf "chains" (heavy child = deepest) = max total length covered by k root paths. HLD from a parent array with $p_i < i$: sizes by a reverse loop, no recursion.

5 Strings

Prefix function, Z-function

4-string/kmp_z.cpp

```
// p[i] = longest proper border of s[0..i]. O(n)
// Find t in s: p = pref(t + '#' + s); match ends where
// p[i] == |t| (start = i - 2|t|). Period of s: n - p[n-1].
vi pref(const string& s) {
  int n = sz(s); vi p(n);
  for (int i = 1; i < n; i++) {
    int k = p[i - 1];
    while (k && s[i] != s[k]) k = p[k - 1];
    p[i] = k + (s[i] == s[k]);
  }
  return p; }
// z[i] = longest common prefix of s and s[i..]; z[0] = n.
vi zfunc(const string& s) {
  int n = sz(s); vi z(n);
  if (n) z[0] = n;
  for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z; }
```

Polynomial hashing

4-string/hash.cpp

When: compare substrings in $O(1)$: LCP by binary search, palindrome check (hash of reversed string), counting distinct substrings of one length. Collision probability $\approx n^2/2^{61}$: safe.

```
// Polynomial hash mod 2^61-1 with random base: get(l, r)
// is the hash of s[l, r). Equal hashes => equal (w.h.p.).
using ull = unsigned long long;
const ull HM = (1ULL << 61) - 1;
ull hmul(ull a, ull b) {
  __uint128_t c = (__uint128_t)a * b;
  ull r = (ull)(c & HM) + (ull)(c >> 61);
  return r >= HM ? r - HM : r; }
mt19937_64 hrng(chrono::steady_clock::now()
                  .time_since_epoch().count());
const ull HB = hrng() % (HM - 1000) + 500;
struct Hash {
  vector<ull> h, p;
  Hash(const string& s) : h(sz(s) + 1), p(sz(s) + 1, 1) {
    for (int i = 0; i < sz(s); i++) {
      h[i + 1] = (hmul(h[i], HB) + (unsigned char)s[i]) % HM;
      p[i + 1] = hmul(p[i], HB);
    } }
  ull get(int l, int r) {
    return (h[r] + HM - hmul(h[l], p[r - l])) % HM; }
};
// HM is prime: inverse of a = hpow(a, HM - 2)
ull hpow(ull b, ull e) {
  ull r = 1;
  for (; e; e >>= 1, b = hmul(b, b)) if (e & 1) r = hmul(r, b);
  return r; }
// LCP of two suffixes: binary search on length with get().
// Exact tests on huge numbers (e.g. concyclic points with big
// coordinates): compute the formula in the field mod HM.
```

Manacher

4-string/manacher.cpp

```
// Palindromes. t = "#s0#s1#...#"; d[i] = radius in t.
// For center i in t: longest palindrome in s has length
// d[i] - 1 and starts at (i - d[i] + 1) / 2.
// Number of palindromic substrings = sum of d[i] / 2.
vi manacher(const string& s) {
  string t = "#";
  for (char c : s) t += c, t += '#';
  int n = sz(t); vi d(n);
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = i > r ? 1 : min(d[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && t[i - k] == t[i + k])
      k++;
    d[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
  }
  return d; }
```

Suffix array + LCP

4-string/sufarr.cpp

When: sorted suffixes: find pattern by binary search ($O(|t| \log n)$), longest repeated substring = max lcp, LCS of s, t: SA of $s\#t$, max lcp of adjacent suffixes from different strings, k-th distinct substring.

```
// Suffix array O(n log n) + LCP (Kasai). sa = sorted suffix
// starts; lcp[i] = LCP(suffix sa[i], suffix sa[i + 1]).
// # distinct substrings = n(n+1)/2 - sum(lcp).
vi suffixArray(string s) {
  s += char(0);                    // s must not contain '\0'
  int n = sz(s); vi p(n), c(n), pn(n), cn(n);
  iota(all(p), 0);
  sort(all(p), [&](int a, int b) { return s[a] < s[b]; });
  for (int i = 1; i < n; i++)
    c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
  for (int h = 1; h < n; h <<= 1) {
    for (int i = 0; i < n; i++) pn[i] = (p[i] - h + n) % n;
    vi cnt(n);                     // counting sort by class
    for (int i = 0; i < n; i++) cnt[c[i]]++;
    for (int i = 1; i < n; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--)
      p[--cnt[c[pn[i]]]] = pn[i];
    cn[p[0]] = 0;
    for (int i = 1; i < n; i++) {
      pii a{c[p[i]], c[(p[i] + h) % n]};
      pii b{c[p[i - 1]], c[(p[i - 1] + h) % n]};
      cn[p[i]] = cn[p[i - 1]] + (a != b);
    }
    c.swap(cn);
    if (c[p[n - 1]] == n - 1) break;
  }
  p.erase(p.begin());
  return p; }
vi lcpArray(const string& s, const vi& sa) {
  int n = sz(s); vi rk(n), lcp(max(0, n - 1));
  for (int i = 0; i < n; i++) rk[sa[i]] = i;
  for (int i = 0, k = 0; i < n; i++) {
    if (rk[i] == n - 1) { k = 0; continue; }
    int j = sa[rk[i] + 1];
    while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
    lcp[rk[i]] = k;
    if (k) k--;
  }
  return lcp; }
```

Aho–Corasick

4-string/aho.cpp

When: many patterns in one text; DP over automaton states (count strings of length n avoiding all patterns: states × length, forbid states with `out > 0`).

```
// Aho-Corasick over 'a'..'z'. After build(): nx = full
// automaton, link = suffix link, out[u] = # patterns that
```

```
// end at u or at its suffix links. O(total * 26)
struct Aho {
  static const int A = 26;
  vector<array<int, A>> nx; vi link, out, term, dict;
  Aho() { newNode(); }
  int newNode() { nx.emplace_back(); nx.back().fill(-1);
    link.push_back(0); out.push_back(0); term.push_back(0);
    return sz(nx) - 1; }
  int add(const string& s) {        // returns end node
    int u = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (nx[u][c] < 0) { int w = newNode(); nx[u][c] = w; }
      u = nx[u][c];
    }
    out[u]++; term[u]++; return u; }
  void build() {                    // BFS
    queue<int> q; dict.assign(sz(nx), -1);
    for (int c = 0; c < A; c++)
      if (nx[0][c] < 0) nx[0][c] = 0; else q.push(nx[0][c]);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      out[u] += out[link[u]];
      if (u) dict[u] = term[link[u]] ? link[u] : dict[link[u]];
      for (int c = 0; c < A; c++) {
        int v = nx[u][c];
        if (v < 0) nx[u][c] = nx[link[u]][c];
        else { link[v] = nx[link[u]][c]; q.push(v); }
      }
    } }
};
// dict[u] = nearest proper suffix-link ancestor where a
// pattern ends (-1 if none): list all matches at a text
// position: if term[u] u itself, then dict[u], dict[dict[u]],
// ... until -1 (term > 0 for each listed node).
// total matches in text: u = 0; for c: u = nx[u][c],
// ans += out[u]. Per-pattern counts: add 1 to cnt[u] for
// each text position, then push cnt along links in reverse
// BFS order.
```

Binary trie (max XOR)

4-string/trie.cpp

```
// Binary trie for max XOR with a stored number (B bits).
struct Trie {
  static const int B = 30;
  vector<array<int, 2>> nx{{0, 0}}; vi cnt{0};
  void add(int x, int d = 1) {      // d = -1 removes
    int u = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1;
      if (!nx[u][b]) {
        nx[u][b] = sz(nx);
        nx.push_back({0, 0}); cnt.push_back(0);
      }
      u = nx[u][b]; cnt[u] += d;
    } }
  int maxXor(int x) {               // trie must be non-empty
    int u = 0, r = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1, w = nx[u][!b];
      if (w && cnt[w] > 0) r |= 1 << i, u = w;
      else u = nx[u][b];
    }
    return r; }
};
```

XOR trie with a bound

4-string/xortrie.cpp

When: DP over prefix XORs: "best dp_j with $x \oplus y_j \le k$"; counting pairs with XOR $\le k$ (store counts).

```
// Binary trie with a min per node: min of val over stored y
// with (x ^ y) <= k. Count version: store counts, add them up.
// Walk: where k has bit 1, the whole subtree with xor bit 0 is
// valid (take its aggregate), then continue on xor bit 1.
struct XorTrie {
  static const int B = 30;
  vector<array<int, 2>> nx{{0, 0}}; vector<ll> mn{LLONG_MAX};
  void add(int y, ll v) {
    int u = 0; mn[0] = min(mn[0], v);
    for (int i = B - 1; i >= 0; i--) {
      int b = y >> i & 1;
      if (!nx[u][b]) {
        nx[u][b] = sz(nx); nx.push_back({0, 0});
        mn.push_back(LLONG_MAX);
      }
      u = nx[u][b]; mn[u] = min(mn[u], v);
    } }
  ll query(int x, int k) {       // LLONG_MAX if no such y
    ll r = LLONG_MAX; int u = 0;
    for (int i = B - 1; i >= 0; i--) {
      int xb = x >> i & 1;
      if (k >> i & 1) {
        if (nx[u][xb]) r = min(r, mn[nx[u][xb]]);
        u = nx[u][!xb];
      } else u = nx[u][xb];
      if (!u) return r;
    }
    return min(r, mn[u]); }
};
```

Counting strings by prefix function

4-string/prefcount.cpp

```
// Number of strings over k letters whose prefix function is
// exactly p (p must be a valid prefix function). O(n).
// D[b] = # distinct letters forbidden after border b.
ll countByPrefix(const vi& p, ll k) {
  int n = sz(p); vi D(n);
  D[0] = 1;
  for (int b = 1; b < n; b++) D[b] = D[p[b - 1]] + (p[b] == 0);
  ll r = k % MOD;                        // first letter: k ways
  for (int i = 1; i < n; i++) if (p[i] == 0)
    r = r * ((k - D[p[i - 1]]) % MOD + MOD) % MOD;
  return r; }
```

Suffix automaton

4-string/sam.cpp

When: everything about all substrings in $O(n)$: distinct count, occurrences of each, LCS of two strings (walk t on SAM(s), on mismatch follow `link`, keep current length), smallest cyclic shift (SAM of $s+s$).

```
// Suffix automaton of s: <= 2|s| states, every substring of s
// = a path from state 0. len = longest string in the state,
// link = suffix link (tree rooted at 0). O(26 |s|) memory.
// - # distinct substrings = sum over states v > 0 of
//   len[v] - len[link[v]]
// - occurrences: cnt[v] = 1 for states created as `cur` (not
//   clones); then, in decreasing len order,
//   cnt[link[v]] += cnt[v].
// - s contains t: walk nx from state 0.
struct SAM {
  vector<array<int, 26>> nx; vi len, link; int last = 0;
  SAM() { newState(0, -1); }
  int newState(int l, int lk) {
    nx.emplace_back(); nx.back().fill(-1);
    len.push_back(l); link.push_back(lk);
    return sz(len) - 1; }
  void extend(int c) {                   // c in [0, 26)
    int cur = newState(len[last] + 1, 0), p = last;
    while (p >= 0 && nx[p][c] < 0) nx[p][c] = cur, p = link[p];
    if (p >= 0) {
      int q = nx[p][c];
      if (len[p] + 1 == len[q]) link[cur] = q;
      else {
        int cl = newState(len[p] + 1, link[q]); nx[cl] = nx[q];
        while (p >= 0 && nx[p][c] == q)
          nx[p][c] = cl, p = link[p];
        link[q] = link[cur] = cl;
      }
    }
    last = cur; }
};
```

Ideas

- LCP of suffixes i, j = min of `lcp` on $[\mathrm{rk}_i, \mathrm{rk}_j)$ (sparse table).
- Minimal rotation of s: suffix array / Booth on $s+s$. s is a power of a word of length $n - p_{n-1}$ iff that divides n.
- Count occurrences of every prefix: `cnt[p[i]]++`, then for i from n down: `cnt[p[i-1]] += cnt[i]`, then `cnt[i]++`.
- Palindromic subsequences: $dp[l][r] = dp[l+1][r] + dp[l][r-1] - dp[l+1][r-1]$, plus $dp[l+1][r-1] + 1$ if $s_l = s_r$ (counts occurrences).

6 Geometry

Point

5-geom/point.cpp

```
// Exact integer geometry: |coord| <= 1e9 -> cross fits in ll.
// For real coordinates use T = long double and compare with
// EPS (e.g. 1e-9): sgn(x) = (x > EPS) - (x < -EPS).
template <class T> struct Pt {
  T x, y;
  Pt operator+(Pt o) const { return {x + o.x, y + o.y}; }
  Pt operator-(Pt o) const { return {x - o.x, y - o.y}; }
  Pt operator*(T k) const { return {x * k, y * k}; }
  Pt operator/(T k) const { return {x / k, y / k}; }
  T dot(Pt o) const { return x * o.x + y * o.y; }
  T cross(Pt o) const { return x * o.y - y * o.x; }
  T cross(Pt a, Pt b) const {
    return (a - *this).cross(b - *this); }
```

```
  T norm2() const { return x * x + y * y; }
  long double len() const { return sqrtl(norm2()); }
  bool operator<(Pt o) const {
    return tie(x, y) < tie(o.x, o.y); }
  bool operator==(Pt o) const { return x == o.x && y == o.y; }
};
using P = Pt<ll>;
using PD = Pt<long double>;
template <class T> int sgn(T v) { return (v > 0) - (v < 0); }
// a.cross(b, c) > 0 <=> a -> b -> c turns left (CCW).
// Rotate 90 CCW: (-y, x). Angle: atan2l(y, x).
// Normalize direction (dx, dy): divide by gcd(|dx|,|dy|),
// then make the first nonzero coordinate positive.
```

Segments and lines

5-geom/segments.cpp

```
// p on segment [a, b] (inclusive)?
bool onSeg(P p, P a, P b) {
  return p.cross(a, b) == 0 && (a - p).dot(b - p) <= 0; }
// do closed segments [a,b] and [c,d] intersect?
bool segInter(P a, P b, P c, P d) {
  int d1 = sgn(a.cross(b, c)), d2 = sgn(a.cross(b, d));
  int d3 = sgn(c.cross(d, a)), d4 = sgn(c.cross(d, b));
  if (d1 * d2 < 0 && d3 * d4 < 0) return true;
  return onSeg(c, a, b) || onSeg(d, a, b) ||
         onSeg(a, c, d) || onSeg(b, c, d); }
// intersection point of LINES ab and cd (not parallel)
PD lineInter(PD a, PD b, PD c, PD d) {
  return a + (b - a) * ((c - a).cross(d - c) /
                        (b - a).cross(d - c)); }
// distance from p to segment [a, b]
long double segDist(PD p, PD a, PD b) {
  PD ab = b - a;
  long double t = ab.norm2() == 0 ? 0 :
      max<long double>(0, min<long double>(1,
          (p - a).dot(ab) / ab.norm2()));
  return (p - (a + ab * t)).len(); }
// distance from p to LINE ab = |cross| / |ab|
// exact intersection of LINES ab, cd for integer points:
// false if parallel or not a lattice point. |coord| <= 1e9.
bool lineInterInt(P a, P b, P c, P d, P& res) {
  __int128 den = (b - a).cross(d - c);
  __int128 num = (c - a).cross(d - c);
  if (den == 0) return false;
  __int128 X = a.x * den + (__int128)(b.x - a.x) * num;
  __int128 Y = a.y * den + (__int128)(b.y - a.y) * num;
  if (X % den || Y % den) return false;
  res = {(ll)(X / den), (ll)(Y / den)};
  return true; }
```

Polygons, convex hull

5-geom/polygon.cpp

```
// twice the signed area (> 0 if counter-clockwise)
ll area2(const vector<P>& pg) {
  ll s = 0;
  for (int i = 0, n = sz(pg); i < n; i++)
    s += pg[i].cross(pg[(i + 1) % n]);
  return s; }
// 0 = outside, 1 = inside, 2 = on boundary. O(n), any polygon
int inPoly(const vector<P>& pg, P p) {
  int n = sz(pg); bool in = false;
  for (int i = 0; i < n; i++) {
    P a = pg[i], b = pg[(i + 1) % n];
    if (onSeg(p, a, b)) return 2;
    if ((a.y > p.y) != (b.y > p.y) &&
        ((b - a).cross(p - a) > 0) == (b.y > a.y)) in = !in;
  }
  return in; }
// Convex hull (monotone chain), CCW, no collinear points.
vector<P> hull(vector<P> pts) {
  sort(all(pts)); pts.erase(unique(all(pts)), pts.end());
  if (sz(pts) < 3) return pts;
  vector<P> h(2 * sz(pts)); int k = 0;
  for (int it = 0; it < 2; it++, reverse(all(pts))) {
    int start = k;
    for (P p : pts) {
      while (k >= start + 2 &&
             h[k - 2].cross(h[k - 1], p) <= 0) k--;
      h[k++] = p;
    }
    k--;
  }
  h.resize(k);
  return h; }
// max squared distance between two points (rotating calipers
// on the hull h, CCW)
ll diameter2(const vector<P>& h) {
  int n = sz(h); ll best = 0;
  if (n < 2) return 0;
  for (int i = 0, j = 1 % n; i < n; i++) {
    while ((h[(i + 1) % n] - h[i]).cross(
              h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
    best = max({best, (h[i] - h[j]).norm2(),
                (h[(i + 1) % n] - h[j]).norm2()});
  }
  return best; }
// Pick: A = I + B/2 - 1, B = sum gcd(|dx|, |dy|) over edges.
```

Polar angle sort

5-geom/angle.cpp

```
// Sort vectors by polar angle in [0, 2pi) from the +x axis.
bool upper(P a) { return a.y > 0 || (a.y == 0 && a.x > 0); }
bool angleLess(P a, P b) {
  if (upper(a) != upper(b)) return upper(a);
  return a.cross(b) > 0; }
// sort(all(v), angleLess);  (no zero vectors!)
// Equal angle: !angleLess(a,b) && !angleLess(b,a).
```

Circles

5-geom/circle.cpp

```
// Circle (c, r) intersect line ab. 0, 1 or 2 points.
const long double EPS = 1e-9;
vector<PD> circleLine(PD c, long double r, PD a, PD b) {
  PD ab = b - a, p = a + ab * ((c - a).dot(ab) / ab.norm2());
  long double h2 = r * r - (c - p).norm2();
  if (h2 < -EPS) return {};
  if (h2 < EPS) return {p};
  PD d = ab * (sqrtl(h2) / ab.len());
  return {p - d, p + d}; }
// Two circles (equal circles -> empty: handle separately).
vector<PD> circleCircle(PD a, long double r1, PD b,
                        long double r2) {
  PD d = b - a; long double d2 = d.norm2();
  if (d2 < EPS) return {};
  long double x = (d2 + r1 * r1 - r2 * r2) / 2;
  long double h2 = r1 * r1 - x * x / d2;
  if (h2 < -EPS) return {};
  PD p = a + d * (x / d2);
  if (h2 < EPS) return {p};
  PD perp = PD{-d.y, d.x} * sqrtl(h2 / d2);
  return {p - perp, p + perp}; }
```

Minimum enclosing circle

5-geom/mec.cpp

```
// Minimum enclosing circle (Welzl), expected O(n) thanks to
// the shuffle. Returns {center, radius}.
// center of the circle through a, b, c (not collinear)
PD circumcenter(PD a, PD b, PD c) {
  PD u = b - a, w = c - a, v = u * w.norm2() - w * u.norm2();
  return a + PD{-v.y, v.x} / (2 * u.cross(w)); }
pair<PD, long double> mec(vector<PD> p) {
  shuffle(all(p), mt19937(1));
  PD c = p[0]; long double r = 0, E = 1 + 1e-12;
  auto out = [&](PD q) { return (q - c).len() > r * E; };
  for (int i = 0; i < sz(p); i++) if (out(p[i])) {
    c = p[i]; r = 0;
    for (int j = 0; j < i; j++) if (out(p[j])) {
      c = (p[i] + p[j]) / 2; r = (p[i] - c).len();
      for (int k = 0; k < j; k++) if (out(p[k])) {
        c = circumcenter(p[i], p[j], p[k]);
        r = (p[i] - c).len();
      }
    }
  }
  return {c, r}; }
```

Point in convex polygon, closest pair

5-geom/convex.cpp

```
// Point in convex polygon h (CCW, no collinear vertices, as
// from hull()) in O(log n): 0 outside, 1 inside, 2 boundary.
int inConvex(const vector<P>& h, P p) {
  int n = sz(h);
  if (n == 1) return p == h[0] ? 2 : 0;
  if (n == 2) return onSeg(p, h[0], h[1]) ? 2 : 0;
  if (h[0].cross(h[1], p) < 0 || h[0].cross(h[n - 1], p) > 0)
    return 0;
  int lo = 1, hi = n - 1;      // wedge h[0], h[lo], h[lo+1]
  while (hi - lo > 1) {
    int m = (lo + hi) / 2;
    (h[0].cross(h[m], p) >= 0 ? lo : hi) = m;
  }
  int c = sgn(h[lo].cross(h[lo + 1], p));
  if (c < 0) return 0;
  if (c == 0 || onSeg(p, h[0], h[1]) ||
      onSeg(p, h[0], h[n - 1])) return 2;
  return 1; }
// Closest pair: min squared distance, n >= 2, |coord| <= 1e9.
// Sweep by x, keep points with x within sqrt(best). O(n log n)
```

```
ll closestPair(vector<P> p) {
  sort(all(p), [](P a, P b) { return a.x < b.x; });
  set<pair<ll, ll>> s;                  // (y, x) in the strip
  ll best = LLONG_MAX;
  for (int i = 0, j = 0; i < sz(p); i++) {
    ll d = (ll)sqrtl(best) + 1;
    while (p[i].x - p[j].x > d) s.erase({p[j].y, p[j].x}), j++;
    for (auto it = s.lower_bound({p[i].y - d, LLONG_MIN});
         it != s.end() && it->first <= p[i].y + d; ++it) {
      ll dx = p[i].x - it->second, dy = p[i].y - it->first;
      best = min(best, dx * dx + dy * dy);
    }
    s.insert({p[i].y, p[i].x});
  }
  return best; }
```

Polygon inside polygon

5-geom/polyinside.cpp

```
// Is simple polygon a inside simple polygon b (boundaries may
// touch)? O(n m log). Idea: boundary of a inside b => a inside
// b (b has no holes). Split every edge of a at the vertices of
// b lying on it; a proper crossing => false; each piece lies
// fully inside or outside, so test its midpoint.
// Points are doubled so midpoints stay integer: needs
// |coord| <= 4e8 (else use Pt<__int128>). Convex b: simply
// check all vertices of a with inConvex (O(n log m)).
bool polyInside(vector<P> a, vector<P> b) {
  for (P& v : a) v = v * 2;
  for (P& v : b) v = v * 2;
  int n = sz(a), m = sz(b);
  for (int i = 0; i < n; i++) {
    P s = a[i], t = a[(i + 1) % n];
    vector<P> cut{s, t};
    for (int j = 0; j < m; j++) {
      P c = b[j], d = b[(j + 1) % m];
      if (sgn(s.cross(t, c)) * sgn(s.cross(t, d)) < 0 &&
          sgn(c.cross(d, s)) * sgn(c.cross(d, t)) < 0)
        return false;                       // proper crossing
      if (onSeg(c, s, t)) cut.push_back(c);
    }
    sort(all(cut), [&](P x, P y) {
      return (x - s).dot(t - s) < (y - s).dot(t - s); });
    for (int k = 0; k + 1 < sz(cut); k++)
      if (!inPoly(b, (cut[k] + cut[k + 1]) / 2)) return false;
  }
  return true; }
```

Angular sweep counting

5-geom/anglecount.cpp

When: counting triangles/pairs by angles around every point (acute, obtuse, containing the origin): $O(n^2 \log n)$.

```
// Angular sweep (needs angleLess): for every vertex v sort the
// other points by angle, then two pointers over the doubled
// order. Example: # acute triangles, no 3 collinear,
// O(n^2 log n). A non-acute triangle has exactly ONE angle
// >= 90: count pairs (u, w) with angle(u, v, w) in [90, 180).
ll countAcute(const vector<P>& a) {
  int n = sz(a); ll bad = 0;
  for (int v = 0; v < n; v++) {
    vector<P> d;
    for (int i = 0; i < n; i++)
      if (i != v) d.push_back(a[i] - a[v]);
    sort(all(d), angleLess);
    int m = sz(d), r1 = 0, r2 = 0;  // (i, r1): ccw angle < 180
    for (int i = 0; i < m; i++) {   // [r2, r1): angle >= 90
      r1 = max(r1, i + 1); r2 = max(r2, i + 1);
      while (r1 < i + m && d[i].cross(d[r1 % m]) > 0) r1++;
      while (r2 < r1 && d[i].dot(d[r2 % m]) > 0) r2++;
      bad += r1 - r2;
    }
  }
  return (ll)n * (n - 1) * (n - 2) / 6 - bad; }
```

Formulas

- Triangle: area $= \frac{1}{2}|ab\sin C| = \sqrt{s(s-a)(s-b)(s-c)}$; $R = \frac{abc}{4S}$, $r = \frac{S}{s}$; law of cosines $c^2 = a^2 + b^2 - 2ab\cos C$; $\frac{a}{\sin A} = 2R$.
- Line through p, q: $a = q_y - p_y$, $b = p_x - q_x$, $c = -(ap_x + bp_y)$; distance from r: $|ar_x + br_y + c|/\sqrt{a^2+b^2}$.
- Rotation by θ: $(x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta)$.
- Circle segment area: $\frac{r^2}{2}(\theta - \sin\theta)$. Sphere: $V = \frac{4}{3}\pi r^3$, $S = 4\pi r^2$; cone $V = \frac{1}{3}\pi r^2 h$.
- Pick: $A = I + \frac{B}{2} - 1$. Lattice points on segment: $\gcd(|dx|, |dy|) + 1$.
- Manhattan distance: rotate $(x + y, x - y)$, then Chebyshev (max of coordinate differences).
- Closest pair: sort by x, sweep keeping a set ordered by y within the current best distance d; $O(n \log n)$.
- Points in convex polygon in $O(\log n)$: binary search on the fan from vertex 0.
- Points strictly inside triangle abc (general position, sorted by x: $a < b < c$): $|B_{ab} + B_{bc} - B_{ac}| - [b \text{ below } ac]$, where $B_{ij} = \#\text{points below segment } ij$ (table in $O(n^2 \log n)$ by angular sort + Fenwick). Empty triangles: count with $= 0$.
- Hull with original indices: sort indices by point, run the same chain. Maximisers of $X \cdot x + Y \cdot y$ $(X, Y > 0)$ lie on the upper-right chain.
- Exact integer intersection: `lineInterInt` (___int128, divisibility check); circles/angles with huge coordinates: exact arithmetic mod $2^{61} - 1$ (`hmul`, `hpow`).

7 DP and techniques

DP patterns

6-misc/dp.cpp

When: D&C optimisation: $dp_{new}[i] = \min_{j<i} dp_{old}[j] + C(j, i)$ and the best j is non-decreasing in i (true if $C(a,c) + C(b,d) \le C(a,d) + C(b,c)$ for $a \le b \le c \le d$, e.g. $C =$ cost of a segment that grows superadditively). Splitting into k groups: $O(kn \log n)$.

```
// LIS, strictly increasing, O(n log n). Non-decreasing:
// upper_bound. Length = sz(d).
int lis(const vi& a) {
  vi d;
  for (int x : a) {
    auto it = lower_bound(all(d), x);
    if (it == d.end()) d.push_back(x); else *it = x;
  }
  return sz(d); }
// 0/1 knapsack reachability with bitset (W up to 1e6):
//   bitset<W + 1> b; b[0] = 1; for (w : items) b |= b << w;
// Subset sums with many equal items: split count c into
// 1, 2, 4, ..., rest (binary splitting) -> O(W sqrt(S)/64).

// Subset DP (Hamiltonian path), O(2^n n^2):
//   dp[mask][v] = path through mask ending at v.
//   dp[1<<v][v] = 1; for mask, v in mask, u not in mask:
//     dp[mask | 1<<u][u] |= dp[mask][v] && adj[v][u].
// Submasks of m (all masks: O(3^n) total):
//   for (int s = m; s; s = (s - 1) & m) { ... }  // + s = 0
// SOS: f[m] = sum over submasks s of m of a[s]. O(2^n n)
//   for i in 0..n-1: for m: if (m >> i & 1) f[m] += f[m^1<<i];

// Digit DP: count x in [0, N] with property. Digits of N
// most significant first; memo[pos][state] for !tight only.
//   ll go(int pos, int st, bool tight, bool started) {
//     if (pos == len) return good(st, started);
//     ll &m = memo[pos][st]; if (!tight && started && m >= 0)
//       return m;
//     ll r = 0; int lim = tight ? dig[pos] : 9;
//     for (int d = 0; d <= lim; d++)
//       r += go(pos+1, next(st, d), tight && d == lim,
//               started || d);
//     if (!tight && started) m = r; return r; }
//   Answer on [L, R] = f(R) - f(L - 1).

// Divide & conquer optimisation: dp_new[i] =
// min_{j < i} dp_old[j] + cost(j, i) when opt(i) is monotone.
// O(k n log n). Call solve(1, n, 0, n - 1).
vector<ll> dpOld, dpNew; ll cost(int j, int i);
void solve(int l, int r, int optl, int optr) {
  if (l > r) return;
  int m = (l + r) / 2, best = -1; ll bv = LLONG_MAX;
  for (int j = optl; j <= min(m - 1, optr); j++) {
    ll v = dpOld[j] + cost(j, m);
    if (v < bv) bv = v, best = j;
  }
  dpNew[m] = bv;
  solve(l, m - 1, optl, best); solve(m + 1, r, best, optr); }
```

Knuth optimisation

6-misc/knuth.cpp

```
// Knuth optimisation: dp[l][r] = min_{l <= m < r} dp[l][m] +
// dp[m+1][r] + w(l, r) (merge stones, optimal BST ...) when
// opt[l][r-1] <= opt[l][r] <= opt[l+1][r] (holds if w is
// monotone and satisfies the quadrangle inequality).
// O(n^2) instead of O(n^3). Here dp[i][i] = 0.
ll w(int l, int r);
ll knuth(int n) {
  vector<vector<ll>> dp(n, vector<ll>(n));
  vector<vi> opt(n, vi(n));
  for (int i = 0; i < n; i++) opt[i][i] = i;
  for (int len = 2; len <= n; len++)
    for (int l = 0; l + len - 1 < n; l++) {
      int r = l + len - 1; dp[l][r] = LLONG_MAX;
      int hiM = min(opt[l + 1][r], r - 1);
      for (int m = opt[l][r - 1]; m <= hiM; m++) {
```

```
        ll v = dp[l][m] + dp[m + 1][r] + w(l, r);
        if (v < dp[l][r]) dp[l][r] = v, opt[l][r] = m;
      }
    }
  return dp[0][n - 1]; }
```

Aliens trick (exactly K items) 6-misc/aliens.cpp

```
// ALIENS TRICK (Lagrangian relaxation / "WQS binary search").
// Want f(K) = min cost using EXACTLY K items (segments,
// groups, chosen elements ...). Conditions:
//  1. f is convex in k: f(k+1) - f(k) is non-decreasing.
//     (Check by brute force on small random tests! Usual
//     sources: Monge / quadrangle-inequality partition DP,
//     problems that are min-cost flows, "pick k" greedy.)
//  2. Without the count limit, for a penalty lam per item you
//     can compute g(lam) = min_k f(k) + lam*k fast, AND the
//     SMALLEST k achieving that minimum (break ties by count).
// Then cnt(lam) is non-increasing; take the smallest lam with
// cnt(lam) <= K; answer = g(lam) - lam*K (use K, not cnt!).
// Integer costs => integer slopes => integer lam suffices;
// lam range: [-(max |f(k+1)-f(k)|) - 1, max ... + 1].
// For max-profit: negate, or flip the tie-break and search.
pair<ll, int> solveLam(ll lam);  // {g(lam), smallest count}
ll aliens(int K, ll lo, ll hi) {
  while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (solveLam(mid).second <= K) hi = mid; else lo = mid + 1;
  }
  return solveLam(lo).first - lo * K; }
// Example solveLam (split a into groups, cost(j, i) per group
// + lam, compare pairs {value, count} lexicographically):
//   dp[0] = {0, 0}; dp[i] = min over j < i of
//   {dp[j].first + cost(j, i) + lam, dp[j].second + 1}.
```

Re-rooting 6-misc/reroot.cpp

```
// RE-ROOTING: answer for EVERY root in O(n) total.
// Pass 1 (down): dp over subtrees for root 0.
// Pass 2 (up):   for each child c of v, compute "the tree
// without c's subtree, seen from v" and push it down to c.
//
// (1) Invertible merge (sum, xor, count): remove the child's
// contribution by subtraction. Sum of distances to all nodes:
//   sub[v] = subtree size, ans[0] = sum of depths;
//   ans[c] = ans[v] - sub[c] + (n - sub[c]).
// (2) max/min merge: keep the best AND second best child
// value of v; for child c use the second if c gave the best.
//   e.g. farthest vertex from every v:
//   up[c] = 1 + max(up[v], best child of v other than c).
//
// (3) Generic (any associative merge): prefix/suffix over
// children. T = value of a rooted subtree seen from its
// parent; merge = combine siblings (identity ID);
// lift(x, v) = value of v's subtree when x = merged children.
// Returns res[v] = merge of ALL neighbours of v (seen from v).
// Example below: T = {size, sum of distances to parent}.
using T = pair<ll, ll>;
const T ID = {0, 0};
T merge(T a, T b) { return {a.first + b.first,
                            a.second + b.second}; }
T lift(T x, int v) { return {x.first + 1,
                             x.second + x.first + 1}; }
vector<T> reroot(const vector<vi>& g, int root = 0) {
  int n = sz(g); vi order{root}, par(n, -1);
  for (int i = 0; i < sz(order); i++)       // BFS order
    for (int v : g[order[i]]) if (v != par[order[i]])
      par[v] = order[i], order.push_back(v);
  vector<T> down(n, ID), up(n, ID), res(n);
  for (int i = n - 1; i > 0; i--) {
    int v = order[i];
    down[par[v]] = merge(down[par[v]], lift(down[v], v));
  }
  for (int u : order) {
    vi ch;
    for (int v : g[u]) if (v != par[u]) ch.push_back(v);
    int k = sz(ch); vector<T> pre(k + 1, ID), suf(k + 1, ID);
    for (int i = 0; i < k; i++)
      pre[i + 1] = merge(pre[i], lift(down[ch[i]], ch[i]));
    for (int i = k - 1; i >= 0; i--)
      suf[i] = merge(lift(down[ch[i]], ch[i]), suf[i + 1]);
    res[u] = merge(up[u], pre[k]);
    for (int i = 0; i < k; i++) {      // parent side for ch[i]
      T rest = merge(up[u], merge(pre[i], suf[i + 1]));
      up[ch[i]] = lift(rest, u);
    }
  }
  return res; }   // here: res[v].second = sum of dist from v
```

Binary / ternary search 6-misc/search.cpp

```
// Binary search on the answer: first x in [lo, hi] with
// ok(x) true (ok monotone false..true); hi+1 if none.
ll firstTrue(ll lo, ll hi, function<bool(ll)> ok) {
  hi++;
  while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    ok(mid) ? hi = mid : lo = mid + 1;
  }
  return lo; }
// Reals: for (int it = 0; it < 100; it++) { m = (l+r)/2; ... }
// Ternary search (max of unimodal f on integers):
//   while (hi - lo > 2) {
//     m1 = lo + (hi - lo) / 3; m2 = hi - (hi - lo) / 3;
//     f(m1) < f(m2) ? lo = m1 : hi = m2; }
//   then check every x in [lo, hi].
// Two pointers: for r: add(r); while (bad) remove(l++);
// Parallel binary search: all queries' [lo,hi] at once,
// one sweep per round (log rounds), e.g. with DSU.
```

Bits, random, misc 6-misc/tricks.cpp

```
__builtin_popcountll(x); __builtin_ctzll(x);  // x != 0
__builtin_clzll(x); __lg(x) /* = floor(log2 x) */;
x & -x;          // lowest set bit
mt19937_64 rng(chrono::steady_clock::now()
               .time_since_epoch().count());
shuffle(all(v), rng);
// __int128 output:
string str128(__int128 x) {
  if (x == 0) return "0";
  string s; bool neg = x < 0;
  unsigned __int128 u = neg ? -(unsigned __int128)x : x;
  while (u) s += char('0' + u % 10), u /= 10;
  if (neg) s += '-';
  return string(s.rbegin(), s.rend()); }
// ceil / floor of a / b for b > 0, any sign of a
// (plain a / b rounds toward zero: -7 / 2 == -3):
ll ceilDiv(ll a, ll b) { return a / b + ((a % b) > 0); }
ll floorDiv(ll a, ll b) { return a / b - ((a % b) < 0); }
// Coordinate compression:
//   vi c = a; sort(all(c)); c.erase(unique(all(c)), c.end());
//   x -> lower_bound(all(c), x) - c.begin()
// Print doubles: cout << fixed << setprecision(10) << x;
// Interactive: cout << "? " << x << endl;  (endl flushes);
//   read answer; on -1 exit immediately. Count queries!
// Big recursion depth (1e6): use iterative or BFS order.
```

Numerical integration 6-misc/simpson.cpp

When: area/volume with curved boundaries (union of circles via $\int$ length of the vertical cut). Split at non-smooth points.

```
// Simpson's rule on [a, b] with n (even) intervals; error
// O(h^4). Smooth f: n = 1e6 gives ~1e-12 relative error.
template <class F>
double integrate(F f, double a, double b, int n = 1 << 20) {
  double h = (b - a) / n, s = f(a) + f(b);
  for (int i = 1; i < n; i++)
    s += f(a + i * h) * (i & 1 ? 4 : 2);
  return s * h / 3; }
```

DP optimisations

- Convex hull trick: $dp_i = \min_j(k_j x_i + b_j) \Rightarrow$ Li Chao tree.
- Knuth: $dp[l][r] = \min_{l \le m < r} dp[l][m] + dp[m+1][r] + w(l,r)$ with $opt[l][r-1] \le opt[l][r] \le opt[l+1][r] \Rightarrow O(n^2)$.
- "Aliens" / Lagrangian relaxation: exactly k items with convex answer in $k \Rightarrow$ binary search a penalty λ per item.
- Exchange argument for greedy order: compare a before b vs. b before a and sort by that comparator (must be a strict weak order).
- Tree DP: $dp[v]$ from children; rerooting: second pass from the root passing the "parent side" value down (prefix/suffix of children).
- Small answers/strange constraints ($n \le 20$, $k \le 10$, $\sum \le 10^5$) hint at bitmask, meet-in-the-middle ($2^{n/2}$), or $\sqrt{}$ splitting.
- Digit DP on sums ($A + B = C$ digit by digit): process from the least significant digit with state = carry (+ previous digits if needed); "digit differs from the previous one" = total − same, in $O(1)$ with row/column sums.
- Composing monotone piecewise-linear functions (slopes 0/1, clamps $\min(\max(x,a),b)$): the composition is again a clamp-plus-shift, store $(lo, hi, shift)$; general case: dynamic segment tree over x.

STL reminders and pitfalls

- `s.lower_bound(x)` on `set`/`map` is $O(\log n)$; `lower_bound(all(s), x)` on a set is $O(n)$!
- `accumulate(all(v), 0LL)` (with `0` it sums in `int`). `abs` on `ll`: `llabs` or `abs((ll)x)`.

- `v.size() - 1` is unsigned: `sz(v) - 1`. `(int)sqrt(x)` may be off by one: adjust with `while (r * r > x) r--; while ((r+1)*(r+1) <= x) r++;`.
- `nth_element(v.begin(), v.begin() + k, v.end())`: k-th in $O(n)$. `partial_sum`, `iota`, `__gcd/gcd`, `lcm` (overflow!).
- `multiset::erase(x)` erases ALL copies; use `s.erase(s.find(x))`.
- `priority_queue` is a max-heap; min-heap: `greater<>`. Sorting with a comparator: must be a strict weak order (`<`, never `<=`).
- `map[x]` inserts x; use `count`/`find` to test. `unordered_map`: `reserve` and `SafeHash`.
- 2D prefix sums: $S_{i+1,j+1} = a_{ij} + S_{i,j+1} + S_{i+1,j} - S_{ij}$; difference array for range add: $d_l \mathrel{+}= v$, $d_r \mathrel{-}= v$, then prefix sum.
- Interactive: flush after every query (`endl` or `cout.flush()`); do not use `fread` input; stop on `-1`.